StreetCare AI

Pothole & Street Defect Report Review Platform

Complete End-to-End Workflow, Every Diagram, and Where the Data Comes From at Each Step

Contents: architecture | data sources | database | data cleaning | model training | knowledge base | service start-up | report screening | evidence & predictions | 10-node LangGraph workflow | human gate | decision documents | API & access | dashboard | chat | full data lineage

Wrenford Borough Council - Street Care Office
Prepared from the StreetCare_AI project specification for team discussion

Contents

Contents 2

0. How to read this document 4

1. The big picture 5

1.1 System architecture . . . 5

1.2 The six entry points, in order . . . 5

1.3 Every data source and where it goes . . . 7

2. Step 1 - Database setup (run_database_setup.py) 9

2.1 The six tables . . . 10

3. Step 2 - Cleaning the raw data (run_data_pipeline.py) 12

3.1 Reading a raw file: load_raw_dataset . . . 12

3.2 Three channels, one reading . . . 13

3.3 The report history (repeat model's training data) . . . 14

3.4 The repair logs (crew hours model's training data) . . . 15

3.5 The labelled statement sentences (NLP model's training data) . . . 16

3.6 The evidence: data_quality_report.json . . . 17

4. Step 3 - Training the three models (run_model_training.py) 18

4.1 Feature engineering . . . 18

4.2 The three models . . . 19

4.3 Evaluation . . . 20

5. Step 4 - Building the knowledge base (run_knowledge_indexing.py) 21

5.1 Two ways of reading the store . . . 22

5.2 Reading a defect standard: requirements come from the PDF . . . 22

6. Step 5 - Starting the REST service (run_api_service.py) 24

6.1 Reading the three live files . . . 25

6.2 The report queue order . . . 25

7. Reviewing one report - everything the code establishes (no LLM) 26

7.1 Screening each requirement . . . 27

7.2 Street works, coordination triggers and the assessment . . . 28

7.3 Worked example (Report Rules, section 13) . . . 29

7.4 Reading the attached documents . . . 30

7.5 Turning model output into figures . . . 30

8. The multi-agent LangGraph workflow 32

8.1 What each node reads, does and writes . . . 33

8.2 The seven agents . . . 33

8.3 Original agent-to-agent communication flow (from the specification) . . . 35

8.4 Opening a review - sequence . . . 36

8.5 The human gate - every check before a decision . . . 37

8.6 Deciding a review - sequence . . . 38
8.7 What each decision writes . . . 38
9. The REST service, access control and the dashboard **40**
9.1 Life of every request . . . 40
9.2 Signing in . . . 41
9.3 Endpoints . . . 41
9.4 Roles and pages . . . 42
9.5 The dashboard pages . . . 44
10. The Street Care chat **46**
10.1 What each role may hear about a report . . . 47
10.2 The figure line (privacy) . . . 47
11. End-to-end data lineage - every step in one table **48**
12. Running it and what you should see **49**

0. How to read this document

This document walks through the StreetCare AI project **in the exact order the data travels**: from the raw CSV files and the sixteen council PDFs, through cleaning, model training and indexing, into the running REST service, the LangGraph multi-agent review, the Highways Inspector's decision, the documents each decision writes, the chat, and the dashboard. Every step has a diagram, a plain-English explanation, a yellow box saying **where its data comes from** and a green box saying **what it produces and who reads it next**.

Two ideas run through the whole project and explain most design choices:

- **The code decides, the language model only explains.** Every requirement result, count, trigger, figure and refusal is worked out by ordinary Python. The seven agents (one Gemini call each) are handed those facts and asked to write prose about them - never to calculate.
- **Nothing is hard-coded.** Every path, mapping, pattern and threshold lives in `config/config.py`; every requirement and its figure is read at run time from the defect type's own PDF standard.
- **A human always decides.** The workflow stops at a gate, saves itself to disk, and waits for a Highways Inspector of the right district who did not open the review.

Colour legend used in every diagram

Shape / colour	Meaning
Yellow box	Raw or live input data (CSV files under data/ or data/input/)
Grey box	Configuration (config/config.py, .env) or the 16 PDF documents in data/docs/
Blue box	A Python function / processing step
Teal box	A trained machine-learning model (.pkl)
Purple hexagon	A language model (Gemini) call made by an agent
Orange rounded box	A person / role, or a decision taken by a person
Green box or cylinder	Something saved: a CSV, JSON, model file, database table or checkpoint
Red box	A refusal or error (ValueError, HTTP 401 / 403 / 404 / 409 / 502)
Dotted arrow	A read of reference data, or an optional / secondary path

The project in one paragraph

Residents, businesses, councillors, bus operators and schools report street defects (Carriageway, Footway, Drainage, Vegetation, Street lighting) through the Website, the StreetCare app or the Contact centre. Each channel writes references, units and yes/no answers its own way. StreetCare AI cleans every report into one canonical reading, screens it against its own defect standard (and a linked type's), counts live street works on its street, finds coordination triggers, reads every sentence of the attached documents, estimates repeat likelihood and expected crew hours with two ML models, has five agents prepare a review with a recommended decision, then **stops** for a Highways Inspector. After the decision it writes exactly what that decision calls for: a works order + resident update, an inspection request, or a coordination brief.

1. The big picture

1.1 System architecture

The architecture diagram on the cover shows the four kinds of input on the left (raw training extracts, live input files, the PDF documents, configuration) and how four offline build scripts turn them into four saved artefacts (cleaned CSVs, model files, the vector store, the database). The REST service then loads all of those at start-up, and the dashboard talks only to the service.

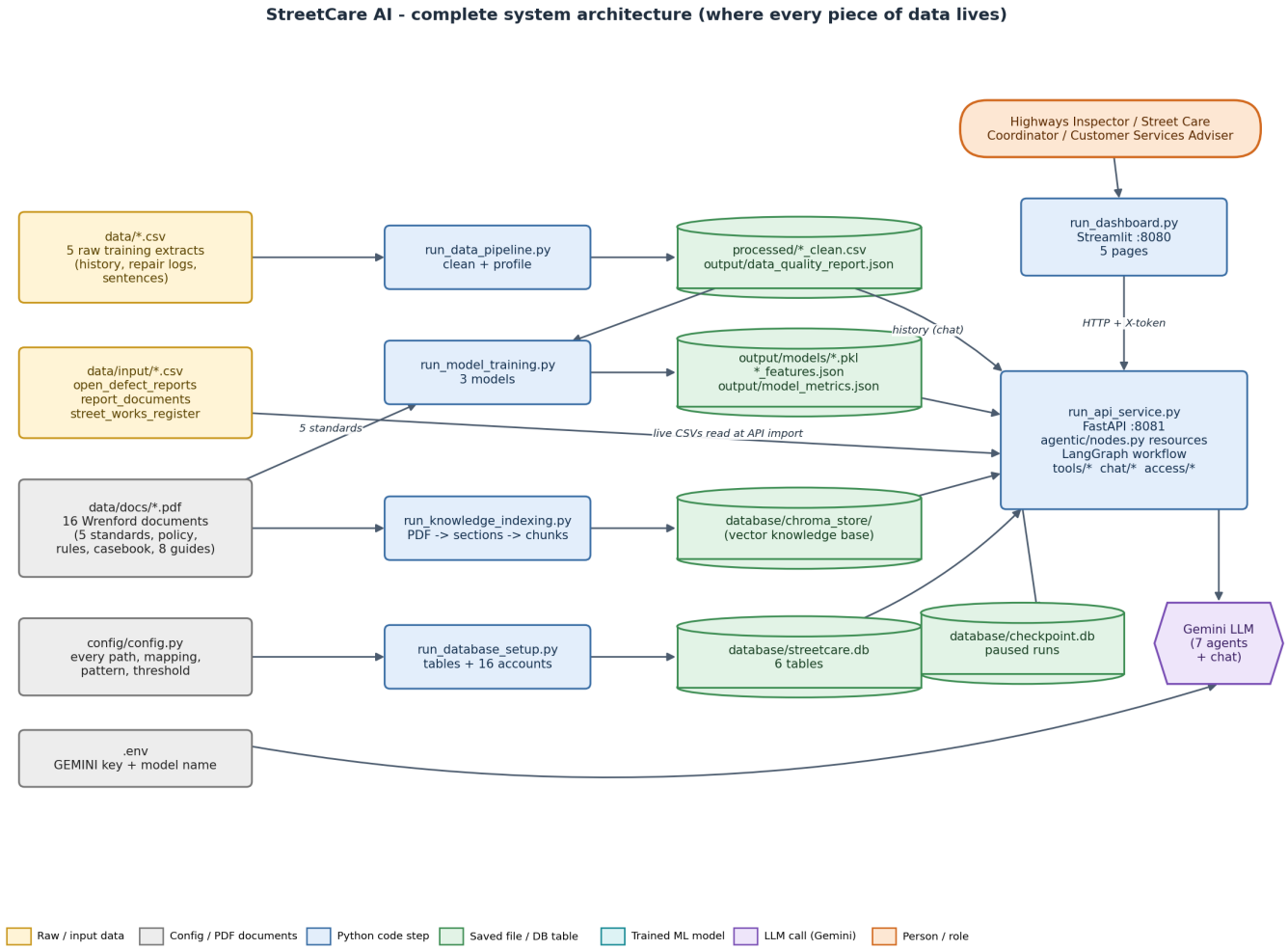


Figure 1 - Complete system architecture: inputs, build stages, saved artefacts, runtime service, LLM and users.

1.2 The six entry points, in order

The project is run from the `Project` folder. Steps 1 to 4 build the system and are run once, in order; steps 5 and 6 start the service and the dashboard and stay running (each in its own terminal). Each step writes files that the next one needs; `run_api_service.py` checks that all of them exist before it starts.

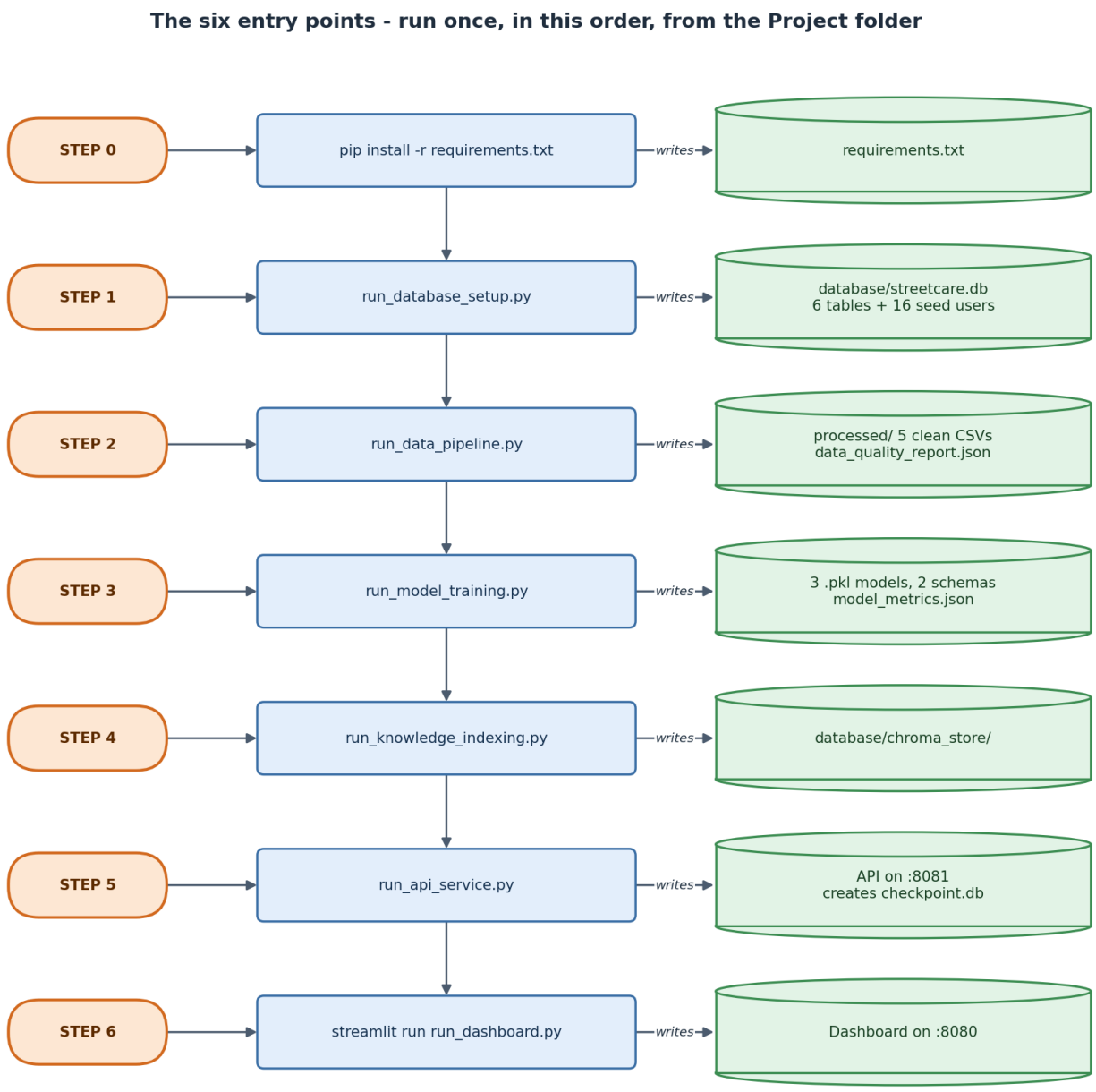


Figure 2 - The six entry points and what each one writes.

Step	Command	Reads	Writes
0	pip install -r requirements.txt	requirements.txt	Python packages
1	python3 run_database_setup.py	config.SEED_USER_ACCOUNTS	database/streetcare.db: 6 tables, 16 accounts
2	python3 run_data_pipeline.py	data/report_history_{train,eval}.csv, data/repair_logs_{train,eval}.csv, data/statement_sentences.csv	processed/*_clean.csv (5), output/data_quality_report.json
3	python3 run_model_training.py	processed/*_clean.csv + data/docs/ (5 standards)	output/models/repeat_model.pkl, crew_hours_model.pkl, sentence_model.pkl, 2 feature schemas, output/model_metrics.json
4	python3 run_knowledge_indexing.py	data/docs/*.pdf (16)	database/chroma_store/ (vector index)
5	python3 run_api_service.py	everything above + data/input/*.csv + .env	API on port 8081; database/checkpoint.db; rows in streetcare.db

Step	Command	Reads	Writes
6	python3 -m streamlit run run_dashboard.py	the API only	Dashboard on port 8080

1.3 Every data source and where it goes

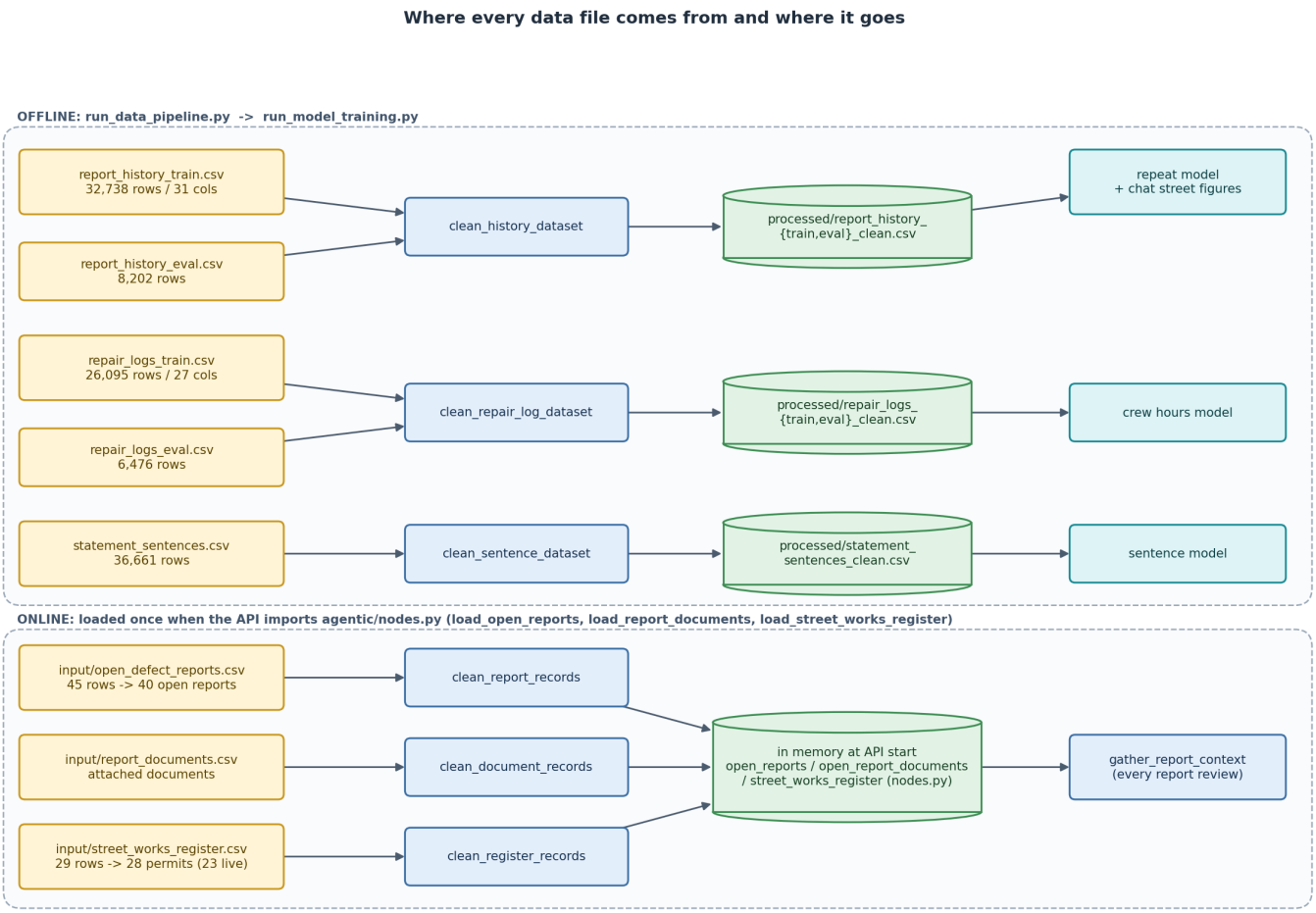


Figure 3 - Offline extracts feed the models; live files feed every report review.

File	Location	As received	What one row is	Cleaned by	Read by
report_history_train.csv	data/	32,738 rows, 31 cols	One revision of a decided report (answer: reported_again)	clean_history_dataset	repeat model (fit); chat street figures
report_history_eval.csv	data/	8,202 rows, same 31 cols	Held-out revisions	clean_history_dataset	repeat model (score only); chat figures
repair_logs_train.csv	data/	26,095 rows, 27 cols	One repaired defect (answer: actual_crew_hours)	clean_repair_log_dataset	crew hours model (fit)
repair_logs_eval.csv	data/	6,476 rows, same 27 cols	Held-out repairs	clean_repair_log_dataset	crew hours model (score only)
statement_sentences.csv	data/	36,661 rows	One labelled sentence of an older report's document	clean_sentence_dataset	sentence (NLP) model
open_defect_reports.csv	data/input/	45 rows, 27 cols -> 40 open reports	One revision of a LIVE report (no answer column)	clean_report_records	every review, queue, detail, chat
report_documents.csv	data/input/	one row per document	A document attached to a live report	clean_document_records	evidence reading

File	Location	As received	What one row is	Cleaned by	Read by
street_works_register.csv	data/input/	29 rows, 6 cols -> 28 permits, 23 live	One street works permit not yet closed	clean_register_records	live/conflicting permit counts
16 PDFs	data/docs/	5 defect standards, Report Rules, Highways Policy, statement guides, casebook	Council documents	pdf_extraction + chunk_splitting	standards, policy sections, chat
config/config.py	config/	supplied complete	Every path, mapping, pattern, threshold, message	-	every module
.env	Project/	GEMINI key + model name	Secrets	-	agentic/language_model.py

2. Step 1 - Database setup (run_database_setup.py)

The first command creates the SQLite database and the sign-in accounts. It runs first because every later stage that writes a record (a run, a review, a decision, an LLM call, a chat turn) needs the tables to exist, and nobody can use the dashboard without an account.

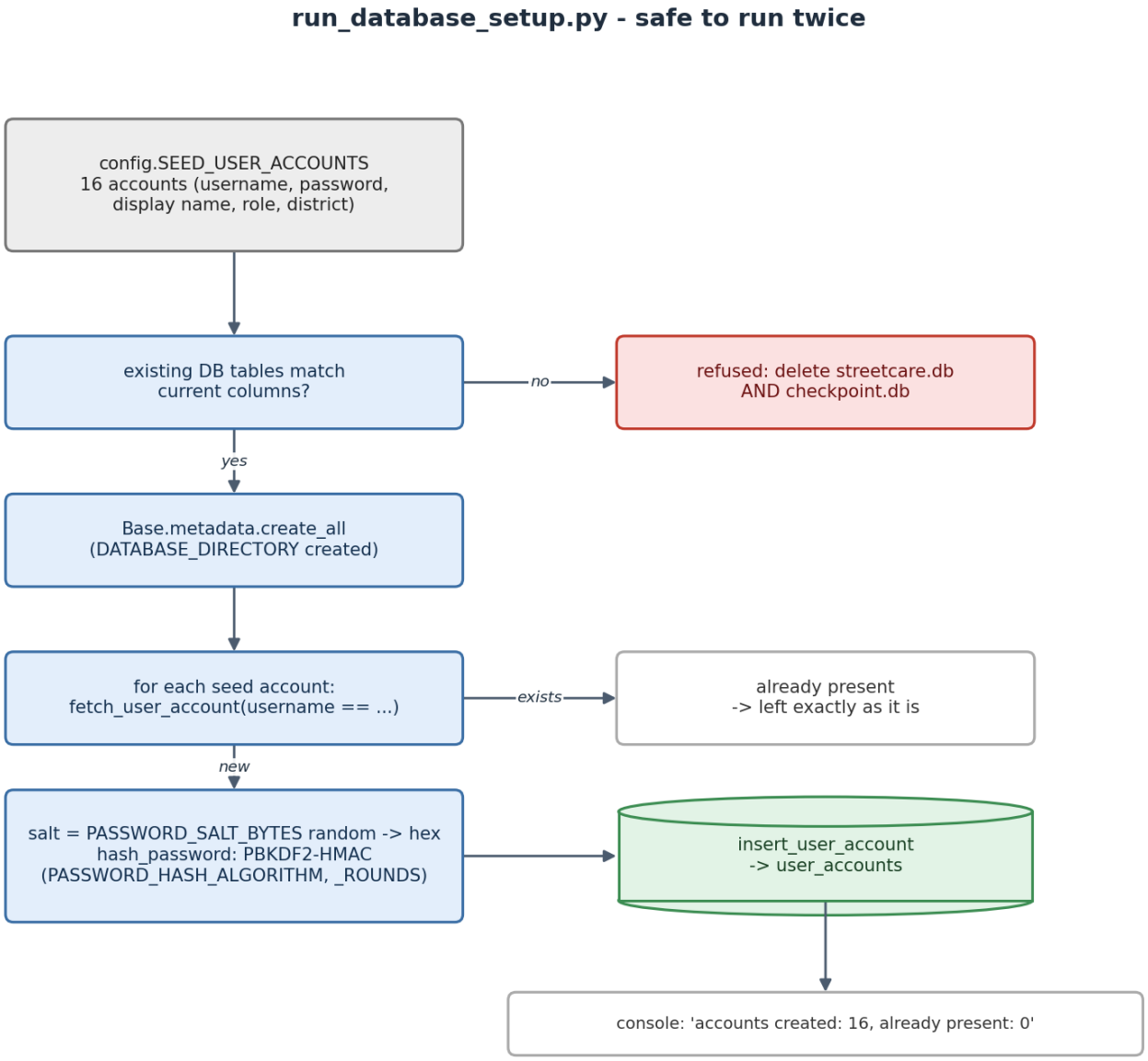


Figure 4 - run_database_setup.py: schema check, table creation, seeding 16 accounts with salted PBKDF2 hashes.

Where the data comes from: `config.SEED_USER_ACCOUNTS` (username, password, display name, role, district for each of 16 people - two Highways Inspectors per district for four districts, plus Street Care Coordinators and Customer Services Advisers). Random salt from `config.PASSWORD_SALT_BYTES`.

Step by step

- **Schema check.** If a database from an earlier version exists and any table's columns differ (by name or nullability), the run is refused and tells you to delete both the database and `config.CHECKPOINT_DATABASE_FILE`.
- **Create tables.** `Base` from persistence/models.py creates the six tables in `config.DATABASE_DIRECTORY`. Importing persistence/database.py only builds `database_engine` and `session_factory` - it never creates tables itself.

- **Seed accounts.** For each seed account, `fetch_user_account` checks the username. An account already present is left exactly as it is (safe to run twice).
- **Hash passwords.** A new account gets its own random hex salt; `hash_password` runs PBKDF2-HMAC with `PASSWORD_HASH_ALGORITHM` over `PASSWORD_HASH_ROUNDS`. The database never holds a password.
- **Console.** Prints the database file, the tables, each account created and the totals - e.g. *accounts created: 16, already present: 0* the first time, *0 / 16* the second.

What comes out / where it goes: database/streetcare.db with tables agent_runs, chat_turns, llm_calls, report_reviews, review_decisions, user_accounts; 16 rows in user_accounts.

2.1 The six tables

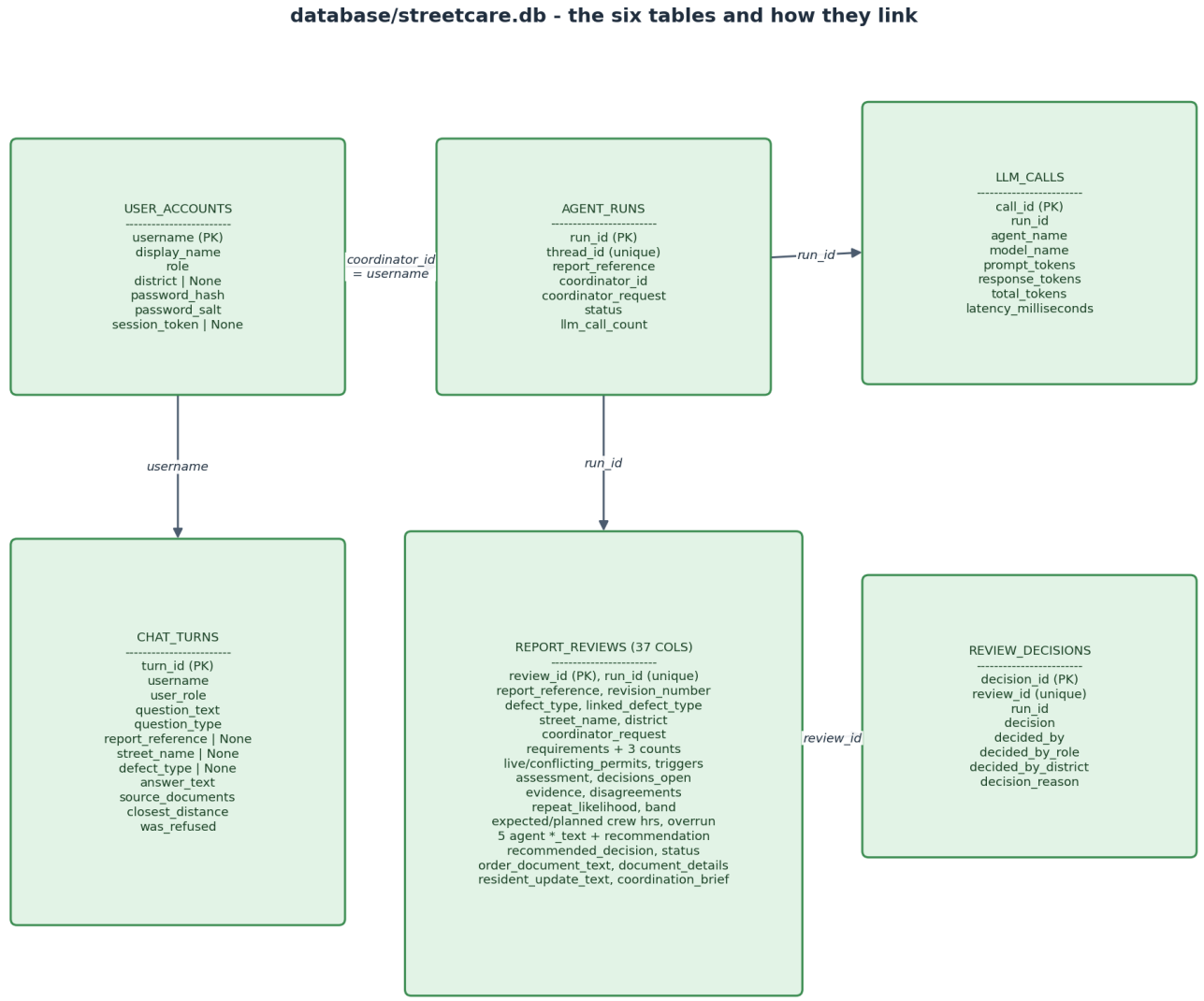


Figure 5 - The six tables and their links. Usernames in agent_runs.coordinator_id and review_decisions.decided_by point back to user_accounts.

Table	Written by	When	Holds
user_accounts	run_database_setup.py; sign_in_user / sign_out_user (token)	setup; each sign-in / sign-out	account, role, district, salted hash, current session token
agent_runs	load_report_context_node; then updated by prepare_..., record_..., report_failed_request	start of each review run	run id, thread id, report, opener, status RUNNING -> AWAITING_DECISION -> COMPLETE / FAILED, call count
llm_calls	every node that calls an agent	right after each agent returns	agent name, model, prompt / response / total tokens, latency ms

Table	Written by	When	Holds
report_reviews	prepare_report_recommendation_node; outcome by record_report_outcome_node	at the gate; after the decision	the whole review (37 columns) and later the documents written
review_decisions	record_report_outcome_node	after the inspector decides	decision, who, role, district, reason (one per review)
chat_turns	ask_street_care_assistant	every chat reply, refusals included	question, type, report, street, defect type, answer, sources, distance, was_refused

No table records a date or time: runs and reviews are listed by report reference, and calls, decisions and chat turns by their own auto-numbered ids. Every write goes through `persistence/repository.py` inside the caller's session, so things that belong together (for example a decision, the review's outcome and the run's COMPLETE status) commit as one transaction.

3. Step 2 - Cleaning the raw data (run_data_pipeline.py)

The five training extracts arrive with real data-quality defects. This stage turns each one into a clean table the models can learn from **and** writes evidence that every repair was needed. It is all-or-nothing: if any extract is refused, no cleaned file and no report is written.

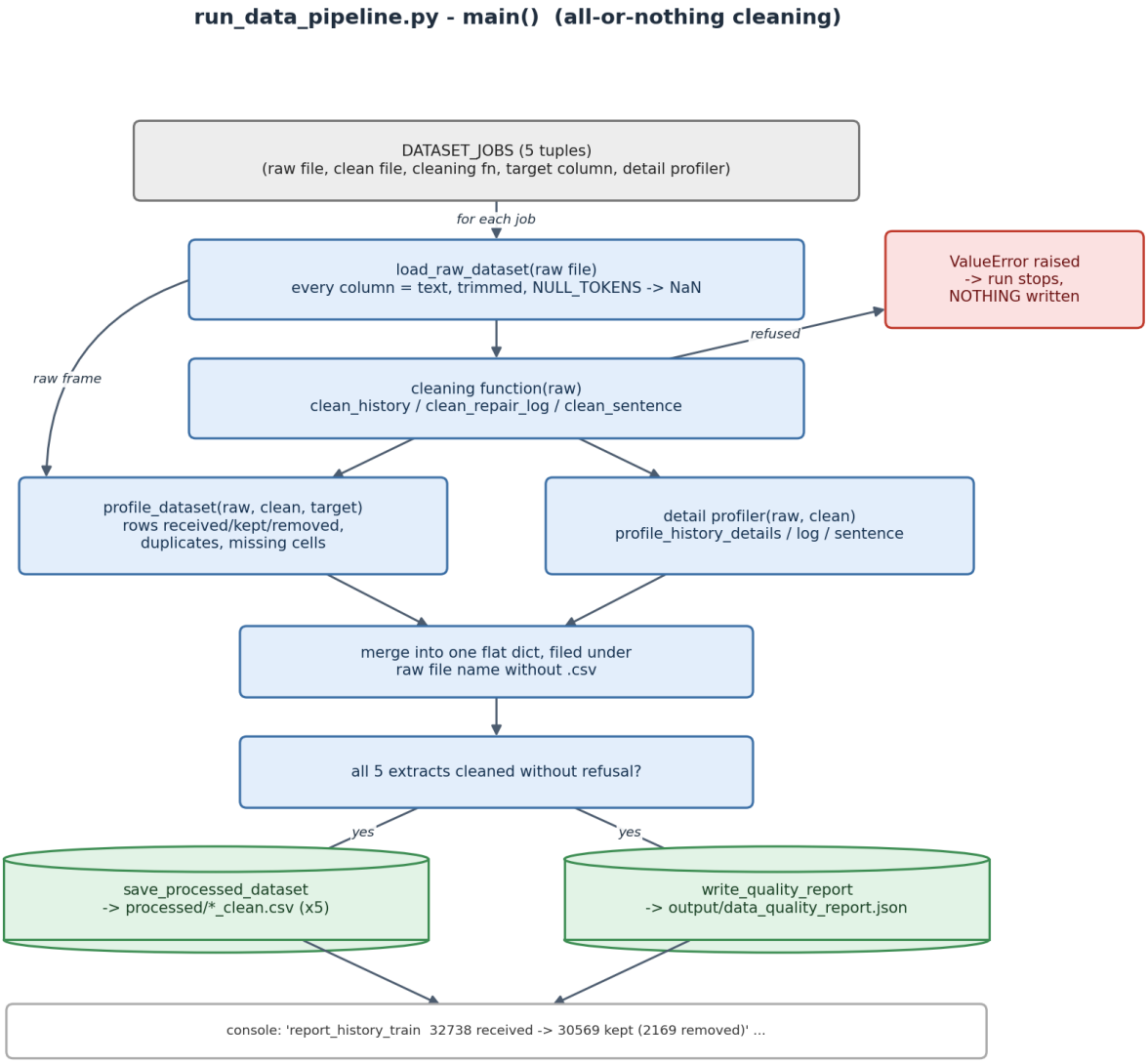


Figure 6 - run_data_pipeline.py main(): five DATASET_JOBS, profiling, all-or-nothing save.

Where the data comes from: `DATASET_JOBS` - five tuples of (raw file, cleaned file, cleaning function, answer column, detail profiler) for report_history_train, report_history_eval, repair_logs_train, repair_logs_eval and statement_sentences.

3.1 Reading a raw file: load_raw_dataset

Every raw file is read **as text** - no column type is guessed. Each value is trimmed, and any value matching one of `config.NULL_TOKENS` (ignoring case and spaces) becomes NaN. This is why 'N/A', 'not known', 'null', 'caller unsure', '-' and empty cells all become the same 'missing'. Only `data_pipeline/dataset_loader.py` reads or writes CSVs.

3.2 Three channels, one reading

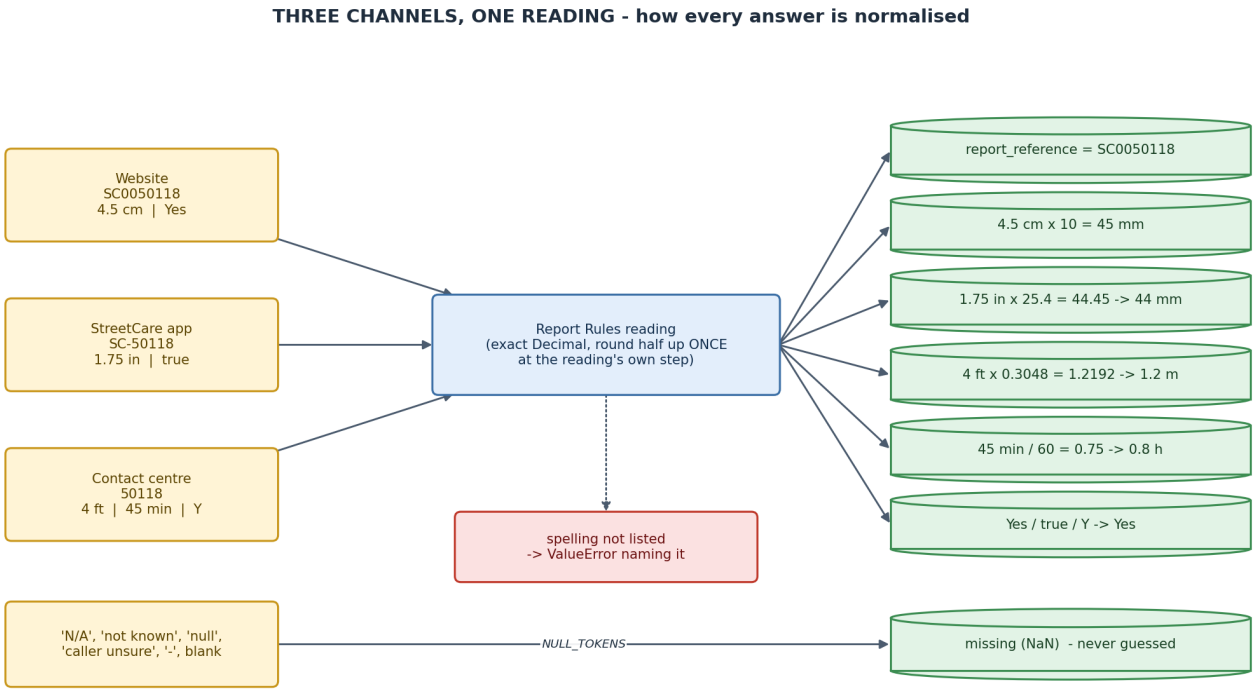


Figure 7 - How the Report Rules turn three channels' notations into one canonical reading.

Every conversion is done as an exact `Decimal` and rounded half up **once**, at the reading's own precision: depths/widths/standing water to whole millimetres, reach/clear height/column height to metres at one decimal, lean to degrees at one decimal, crew time to hours at one decimal. So 4.5 cm = 45 mm, 1.75 in = 44 mm, 4 ft = 1.2 m, 45 min = 0.8 h. A spelling the mappings do not list is refused with a ValueError naming the column and values - it is never guessed.

Defect in the raw data	Example	Repaired by	Config used
Exact duplicate rows	same permit written twice	remove_duplicate_records	-
Reference in 3 notations	SC0050118 / SC-50118 / 50118	convert_report_references	REPORT_REFERENCE_PATTERN, _PREFIX, _DIGITS
Category in channel words / codes / case	'CW', 'carriageway', 'TS', 'Y', 'true'	normalize_category_values	*_VALUE_MAPPINGS
Units mixed	cm / in / ft / min / hrs	convert_readings	FORM_READINGS, LOG_READINGS
Counts as text	'3', revision numbers	convert_whole_numbers	*_WHOLE_NUMBER_PATTERNS, WHOLE_NUMBER_DTYPE
Impossible reading	negative or huge depth	treat_values_below_minimum / above_maximum -> NaN	*_MINIMUM_VALUES, *_MAXIMUM_VALUES
Many revisions of one report	revision 1, 2, 3	keep_latest_revision_of_each_report	REVISION_NUMBER_COLUMN
Missing answer	reported_again blank	remove_rows_missing_target	*_TARGET_COLUMN
Markup / entities / no-break spaces	, &,	convert_markup_to_plain_text	MARKUP_PATTERN
Sentence in CAPITALS	'WATER ACROSS THE ROAD'	lower_text_written_in_capitals	-
Too-short sentence	fewer than N words	dropped in clean_sentence_dataset	MINIMUM_SENTENCE_WORDS

Some gaps are **facts**, not defects: a water depth on a footway report, a column height on a carriageway report, a repeat cause on a defect that never came back. These stay empty in the cleaned tables; the preparation saved inside each model fills them when it fits and when it scores.

3.3 The report history (repeat model's training data)

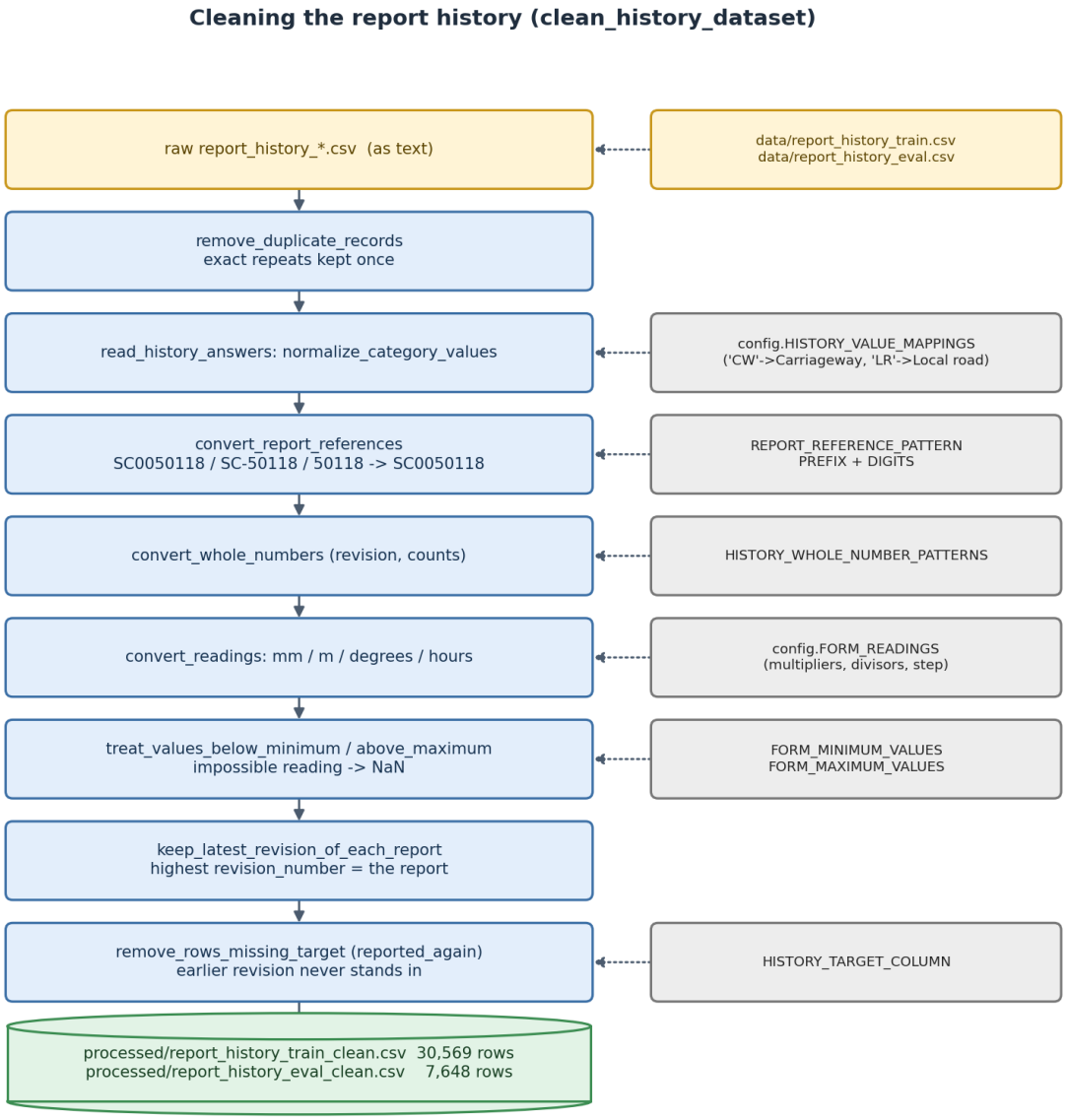


Figure 8 - clean_history_dataset: from 32,738 / 8,202 raw rows to 30,569 / 7,648 clean rows.

The key rule is the **latest revision**: a report sent again with an answer added or corrected carries a higher `revision_number`, and only the highest revision is the report. If that latest revision has no `reported_again` answer, the report is left out - an earlier revision never stands in for it. This count is recorded as `answered_earlier_revisions_removed` in the quality report.

3.4 The repair logs (crew hours model's training data)

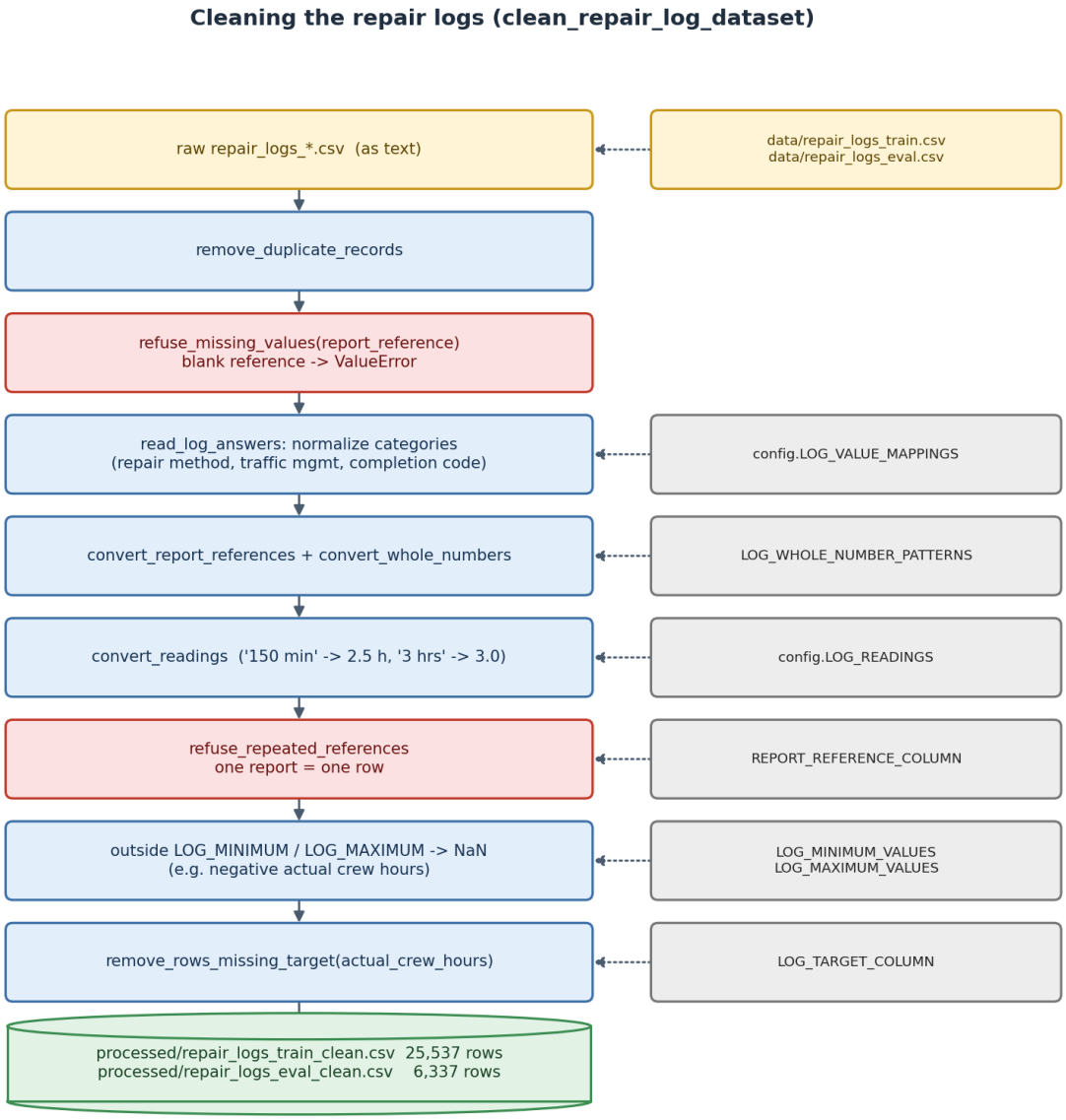


Figure 9 - clean_repair_log_dataset: 26,095 / 6,476 raw rows to 25,537 / 6,337 clean rows.

Repair logs record crew hours in hours, in minutes and as 'hrs'. A blank report reference, or the same report on two rows, is refused outright because each log must belong to exactly one report. A crew-hours value outside the configured limits becomes missing and the row is then dropped (counted as `rows_with_impossible_target`).

3.5 The labelled statement sentences (NLP model's training data)

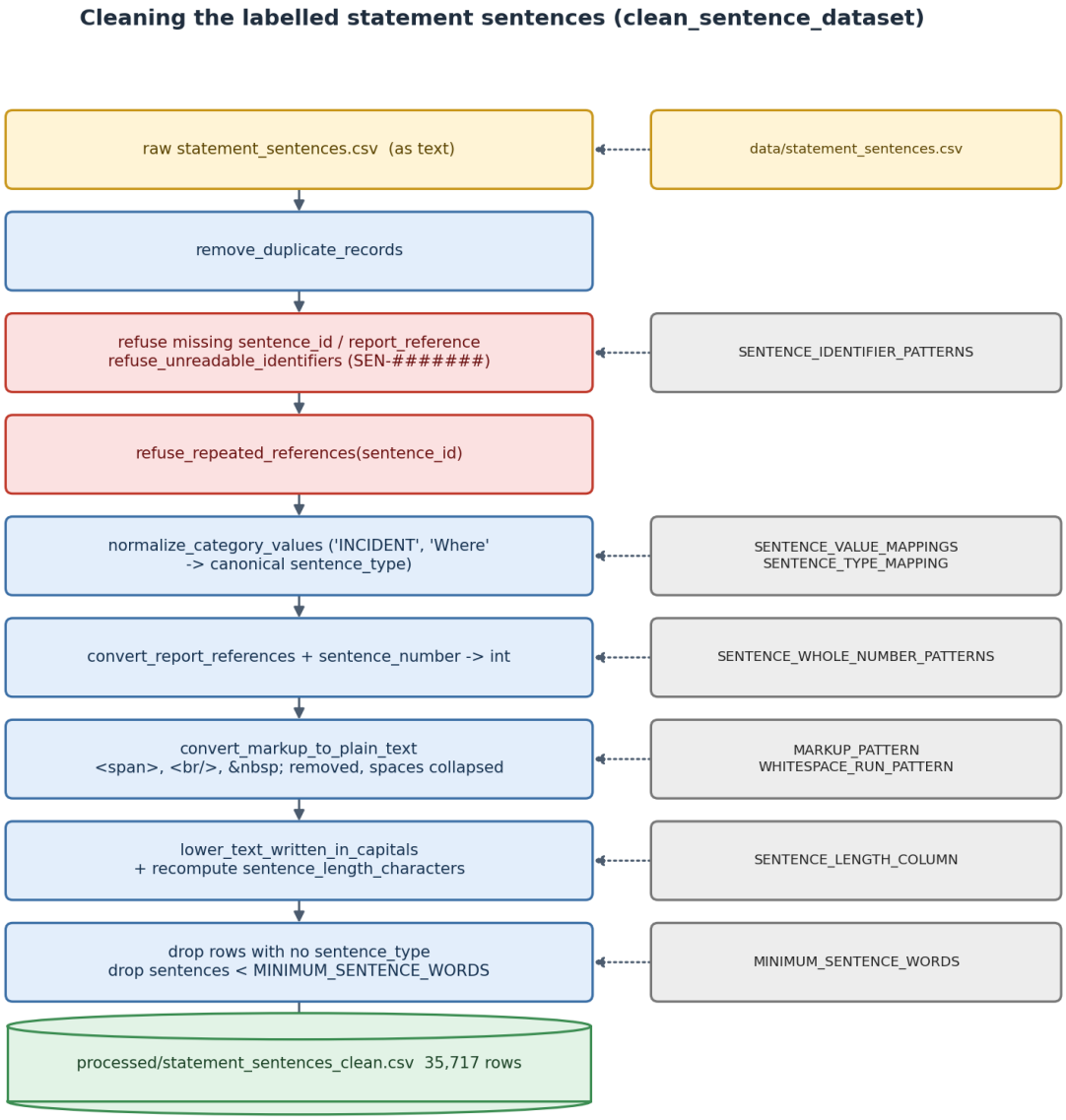


Figure 10 - clean_sentence_dataset: 36,661 raw rows to 35,717 clean sentences.

3.6 The evidence: data_quality_report.json

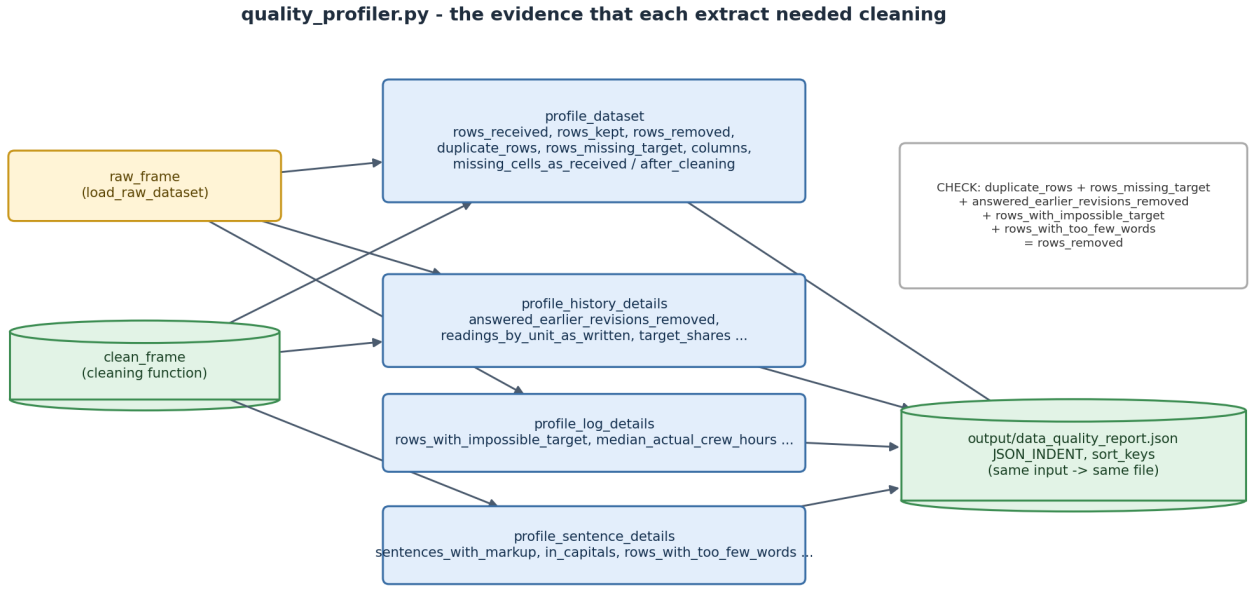


Figure 11 - quality_profiler.py: what each extract looked like before and after cleaning.

For every extract, `profile_dataset` records rows received / kept / removed, duplicates, rows missing the target, column count and missing cells before and after. A detail profiler adds evidence specific to that file (references not written canonically, readings counted by the unit they were written in, values below minimum / above maximum, category spellings not canonical, target shares, sentences with markup or in capitals, median crew hours ...). The reasons for removal always add up to `rows_removed`. The report is JSON, indented and key-sorted, so the same input always gives the same file.

What comes out / where it goes: processed/report_history_train_clean.csv (30,569), processed/report_history_eval_clean.csv (7,648), processed/repair_logs_train_clean.csv (25,537), processed/repair_logs_eval_clean.csv (6,337), processed/statement_sentences_clean.csv (35,717) and output/data_quality_report.json. Next reader: run_model_training.py (and later the chat, for street figures).

4. Step 3 - Training the three models (run_model_training.py)

Three models are trained in one run and saved **only if every one** reaches its configured minimum. If any falls short the run stops, names every shortfall and saves nothing.

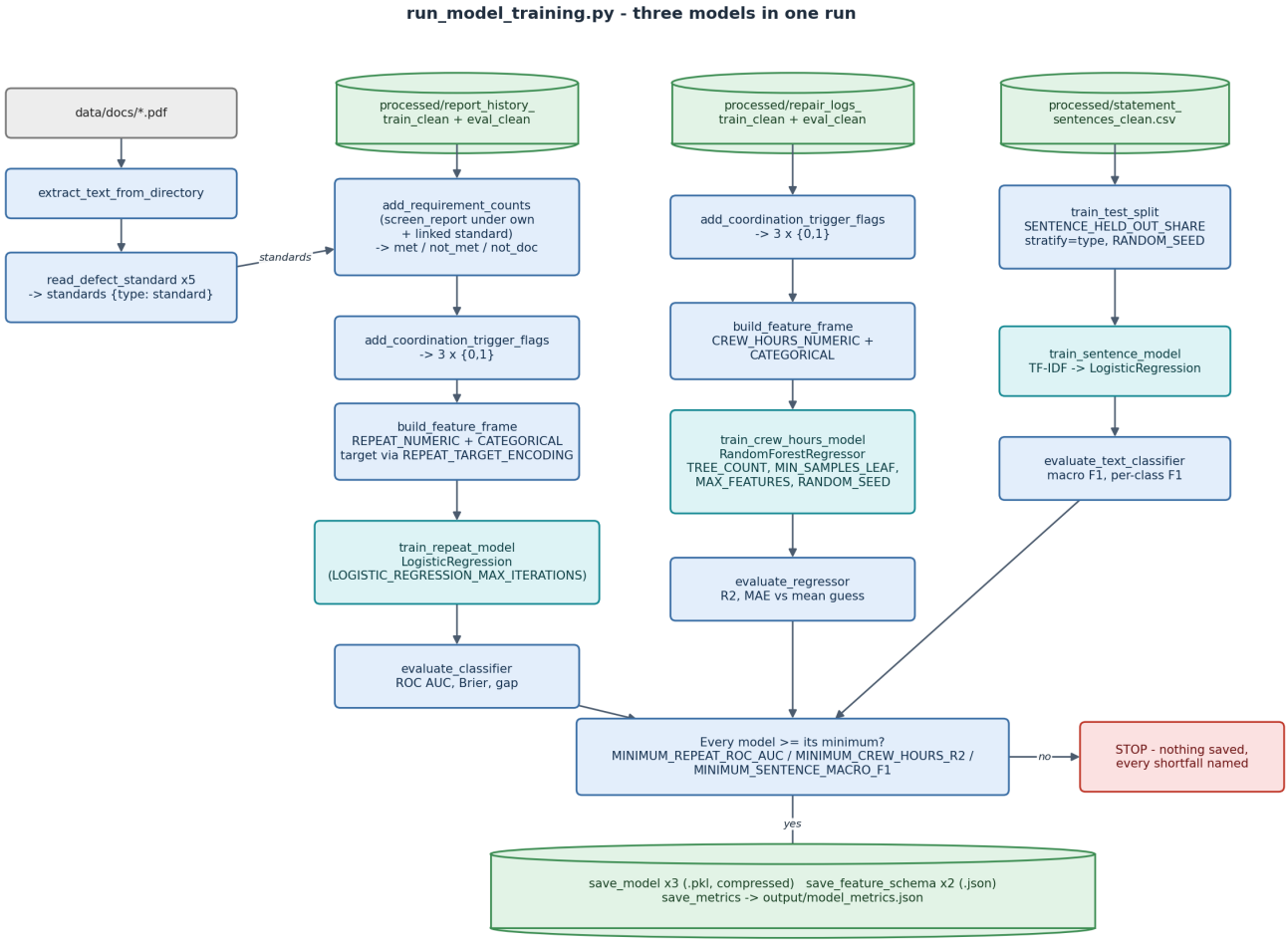


Figure 12 - run_model_training.py: three parallel training paths and one save gate.

Where the data comes from: processed/report_history_train_clean.csv + eval (repeat model); processed/repair_logs_train_clean.csv + eval (crew hours model); processed/statement_sentences_clean.csv (sentence model); data/docs/ - the five defect standards, read with `read_defect_standard` (no vector index needed at this stage).

4.1 Feature engineering

- **add_requirement_counts** - each past report is screened with `screen_report` under its own standard and its linked type's, exactly as a live report will be. Three new columns: requirements Met, Not met, Not documented. This is why the five standards are needed at training time.
- **add_coordination_trigger_flags** - each history row and repair log gets three 0/1 flags from `find_coordination_triggers` (live street works, traffic-sensitive street, high column).
- **build_feature_frame** - lays the inputs out in one fixed order: numeric columns then categorical columns from config. Gaps are NaN (never None, 'nan' or ''). The same function is used at scoring time, so training and live inputs always match.
- **Target encoding** - `reported_again` becomes 0/1 via `config.REPEAT_TARGET_ENCODING`; an unknown answer is refused.

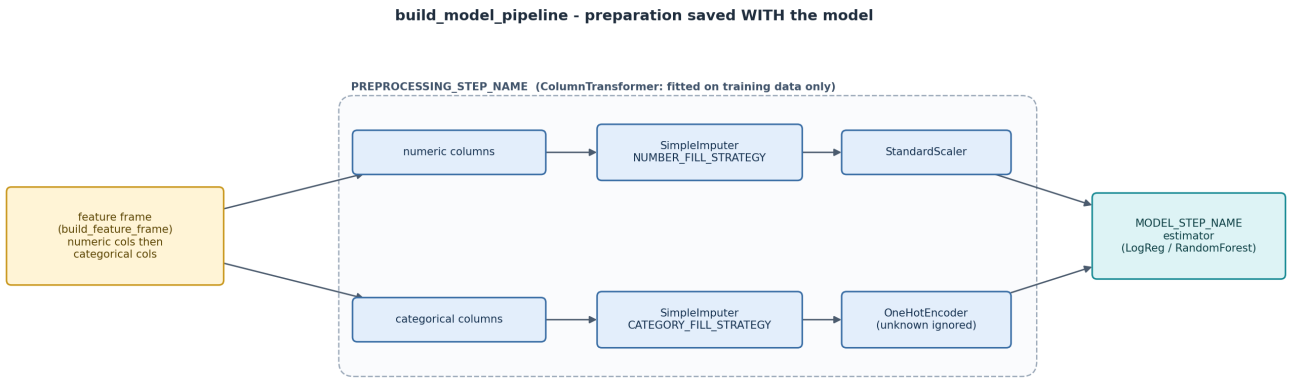


Figure 13 - build_model_pipeline: imputation, scaling and one-hot encoding are saved inside the model.

Because preparation and estimator are bundled into one scikit-learn `Pipeline` (first step `PREPROCESSING_STEP_NAME`, last `MODEL_STEP_NAME`), the saved .pkl can score a raw live report with gaps exactly as it was fitted, without any separate pre-processing code.

4.2 The three models

Model	Algorithm	Learns from	Predicts	Must reach	Behaviour checks
Repeat	LogisticRegression (LOGISTIC_REGRESSION_MAX_ITERATIONS)	history train (scored on history eval)	probability the defect is reported again	MINIMUM_REPEAT_ROC_AUC; mean score within 0.03 of real share	Met -> Not met raises score for >=95% (avg +0.03); +1 conflicting permit raises for >=95% (avg +0.05); live queue spans all 3 bands
Crew hours	RandomForestRegressor (CREW_HOURS_TREE_COUNT, MIN_SAMPLES_LEAF, MAX_FEATURES, RANDOM_SEED)	repair logs train (scored on eval)	actual crew hours	MINIMUM_CREW_HOURS_R2; MAE < 0.75 x mean-guess MAE	no forecast below training minimum; traffic-sensitive street +0.5 h avg for >=90%; +1 conflicting permit +0.25 h avg for >=90%; some overruns, some not
Sentence (NLP)	TF-IDF + LogisticRegression	statement sentences, stratified split SENTENCE_HELD_OUT_SHARE, RANDOM_SEED	one of 9 sentence types	MINIMUM_SENTENCE_MACRO_F1	every type predicted; >=75% recall per type; any text (unseen words too) gets a type

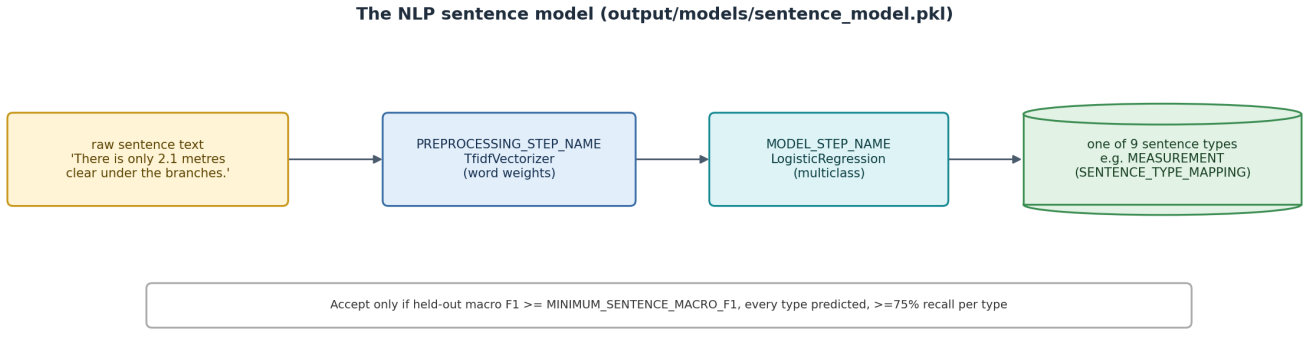


Figure 14 - The NLP sentence model takes raw text and returns one of nine sentence types.

4.3 Evaluation

- **evaluate_classifier** (repeat): train and held-out ROC AUC, generalisation gap, Brier score, mean predicted probability, positive share.
- **evaluate_regressor** (crew hours): train / held-out R squared, gap, MAE, MAE of always guessing the training mean, lowest / highest prediction.
- **evaluate_text_classifier** (sentence): train / held-out macro F1, gap, accuracy, weighted F1, per-class F1.
- All scores are floats rounded to `METRIC_DECIMALS`. No evaluation function ever refits a model.

What comes out / where it goes: output/models/repeat_model.pkl, crew_hours_model.pkl, sentence_model.pkl (compressed with MODEL_FILE_COMPRESSION); output/models/repeat_features.json and crew_hours_features.json (exact column order); output/model_metrics.json (algorithm, training_rows, held_out_rows, scores, minimum_required, scikit_learn_version per model). Next reader: agentic/nodes.py at API start-up.

5. Step 4 - Building the knowledge base (run_knowledge_indexing.py)

The sixteen council PDFs are the **only** source of any requirement, rule or check the system states. This stage cuts them into searchable passages so that every statement can be traced back to the document it came from.

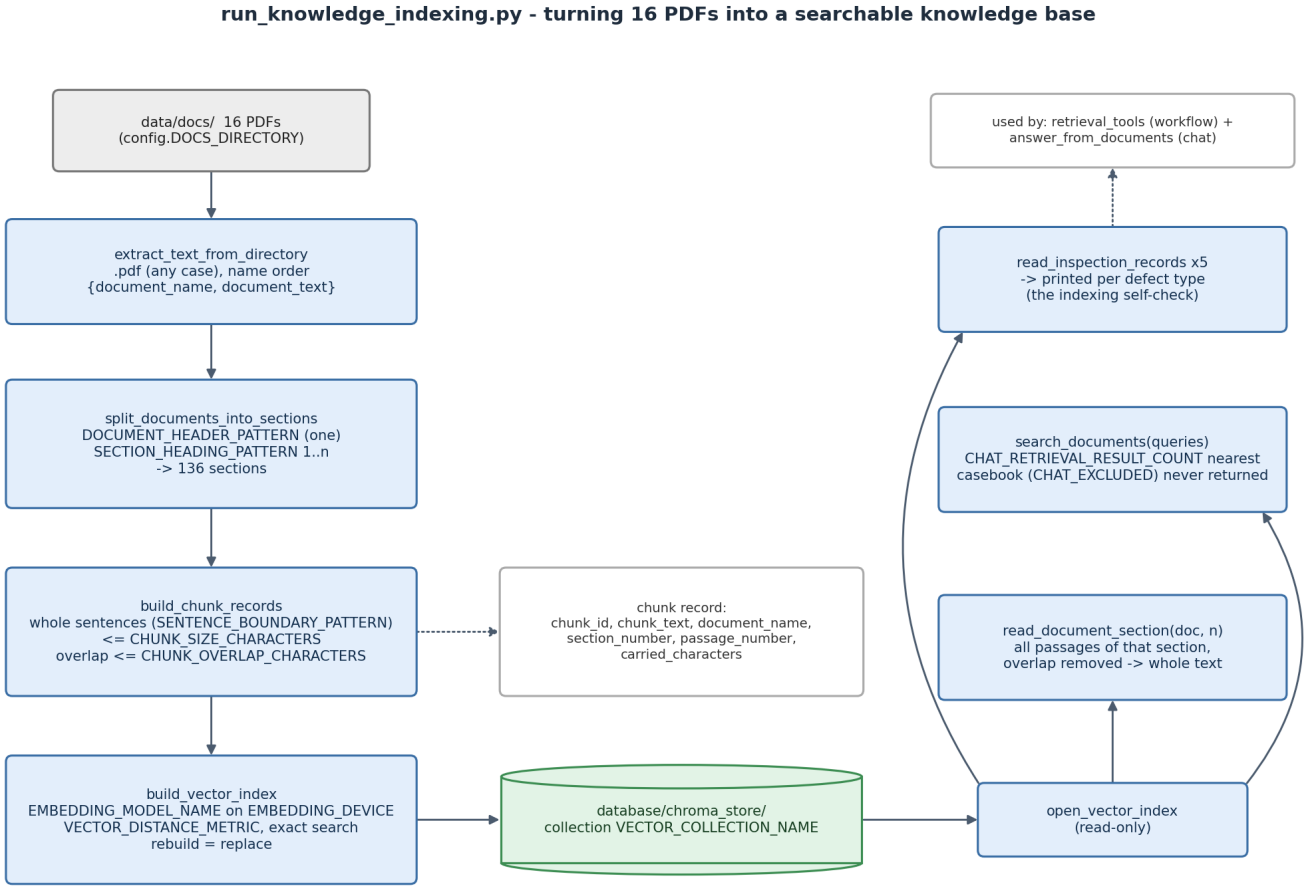


Figure 15 - From 16 PDFs to a persistent Chroma vector store, and the two ways of reading it back.

Where the data comes from: data/docs/*.pdf (config.DOCS_DIRECTORY): five defect standards (Carriageway, Footway, Drainage, Vegetation, Street lighting), the Report Rules, the Highways Policy, statement guides per sentence type, and a casebook that the chat must never quote.

Step by step

- **extract_text_from_directory** - reads every file ending in .pdf (any case) in name order; each page's text joined by one line break. A PDF with no pages or a page with no text is refused.
- **split_documents_into_sections** - every document starts with WRENFORD BOROUGH COUNCIL, its title, one header line (DOCUMENT_HEADER_PATTERN) and sections numbered from 1 (SECTION_HEADING_PATTERN). Any other layout is refused. Result: **136 sections**.
- **build_chunk_records** - each section is cut into passages of whole sentences up to CHUNK_SIZE_CHARACTERS, never spanning two sections; each passage after the first repeats up to CHUNK_OVERLAP_CHARACTERS of whole sentences from the end of the previous one, so a sentence at a boundary is always whole somewhere. chunk_id is built from the document name and passage number, so identical documents always give identical ids.

- **build_vector_index** - embeds each passage with EMBEDDING_MODEL_NAME on EMBEDDING_DEVICE into collection VECTOR_COLLECTION_NAME, compared by VECTOR_DISTANCE_METRIC, with exact (not approximate) nearest-neighbour search. Rebuilding replaces the store; if the embedding model fails to load, the old store is left untouched.
- **Read-back self-check** - the store is reopened and section INSPECTION_RECORDS_SECTION_NUMBER of each of the five standards is printed under its defect type. Two defect types reading back the same section would reveal a broken index.

5.1 Two ways of reading the store

Function	Used by	What it returns
read_document_section(collection, document, section)	tools/retrieval_tools.py (workflow + chat)	One whole section of one named document, rebuilt from its passages with the overlap removed - every sentence once
search_documents(collection, query_texts)	answer_from_documents (chat only)	CHAT_RETRIEVAL_RESULT_COUNT nearest passages per query, with distance; casebook (CHAT_EXCLUDED_DOCUMENTS) never returned

5.2 Reading a defect standard: requirements come from the PDF

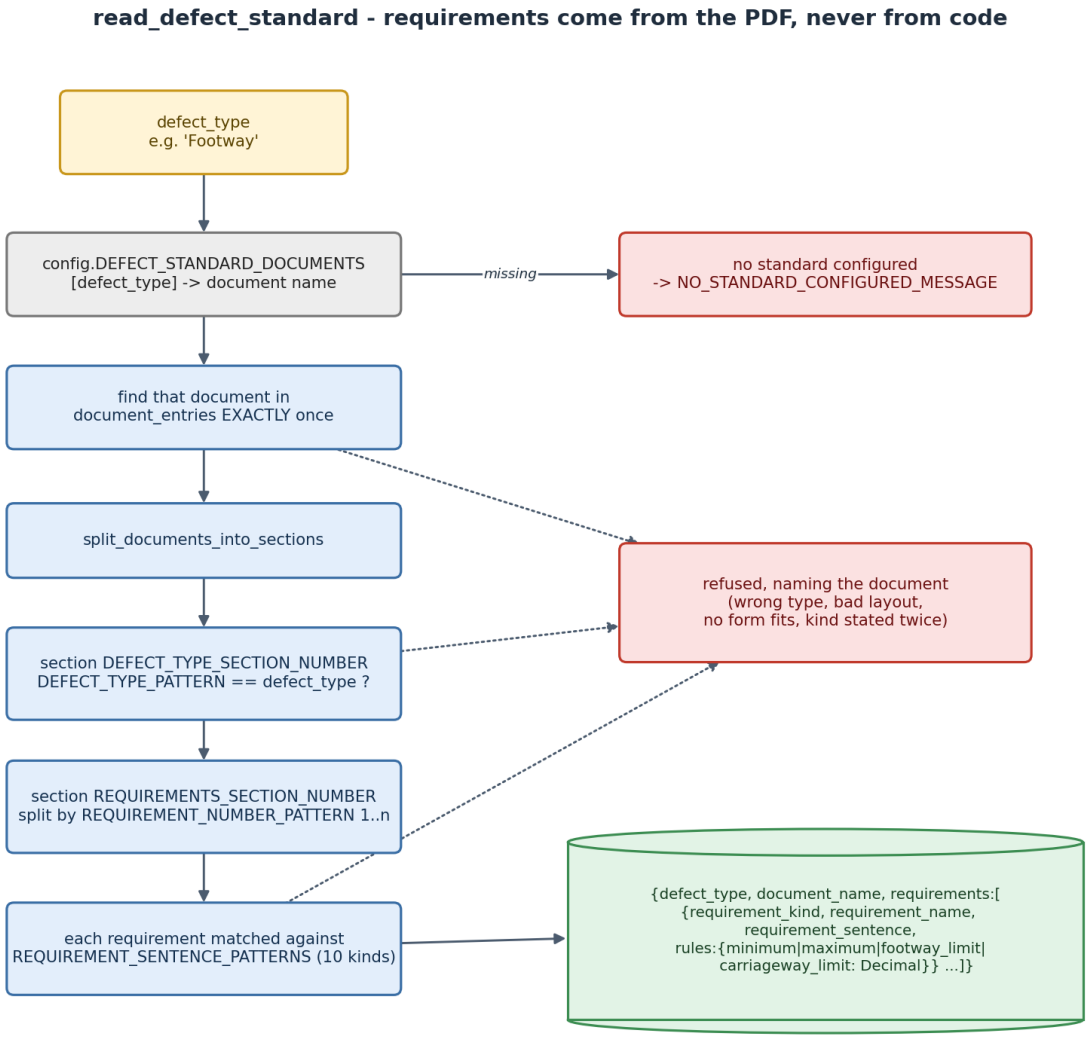


Figure 16 - read_defect_standard: how a defect type's requirements and figures are read from its own PDF.

This module is what keeps the requirements out of the code. The carriageway and footway standards word their depth requirement alike but set **different figures**, so a requirement is only ever read from the report's own standard. Each

requirement must fit exactly one of the ten sentence forms in REQUIREMENT_SENTENCE_PATTERNS (from LOCATION_FIXED_REQUIREMENT to COLUMN_LEAN_REQUIREMENT); the figures inside it (minimum, maximum, footway limit, carriageway limit) are kept as exact Decimals.

What comes out / where it goes: database/chroma_store/ (the knowledge base). Console: '16 documents, 136 sections, NNN passages indexed in database/chroma_store' followed by each standard's inspection-records section.

6. Step 5 - Starting the REST service (run_api_service.py)

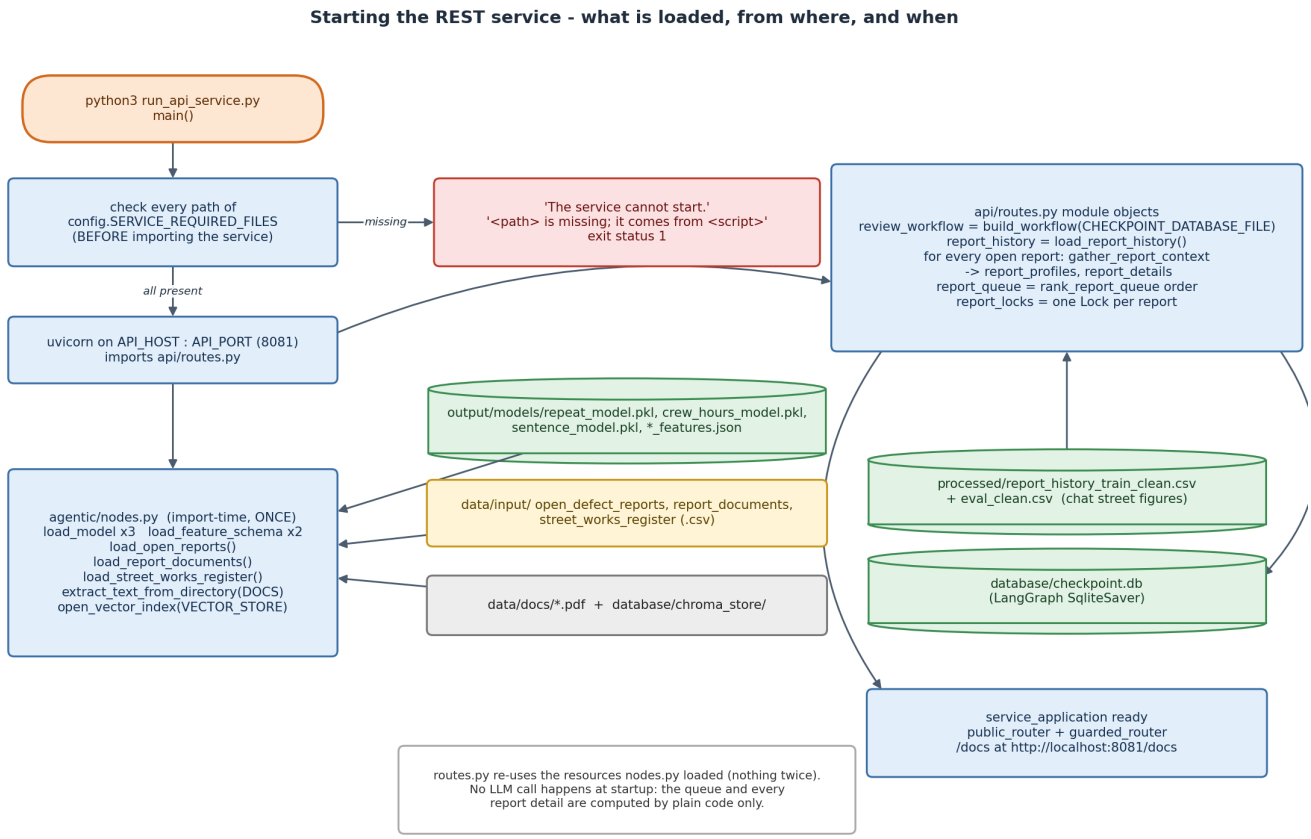


Figure 17 - Service start-up: file check, import-time loading of every resource, pre-computing the queue.

Step by step

- **File check first.** Every path in SERVICE_REQUIRED_FILES (2 cleaned history files, 3 models, 2 schemas, 3 live files, the docs folder, the store's VECTOR_STORE_FILE_NAME and the database) is checked *before* anything is imported. A missing one prints e.g. *output/models/repeat_model.pkl is missing; it comes from run_model_training.py* and exits with status 1.
- **agentic/nodes.py resources** are loaded once at import: repeat_model, crew_hours_model, sentence_model, the two feature schemas, open_reports, open_report_documents, street_works_register, knowledge_documents (16 PDFs' text) and knowledge_collection (the Chroma store).
- **api/routes.py objects**: the compiled workflow with its SQLite checkpointer; report_history (cleaned train + eval history, never the live queue); every queued report's profile and detail computed with gather_report_context; the queue order from rank_report_queue; one threading lock per report.
- **No LLM call happens at start-up.** The queue and every detail come from the same tools the workflow uses, so the queue, the detail page and the stored review always agree.

6.1 Reading the three live files

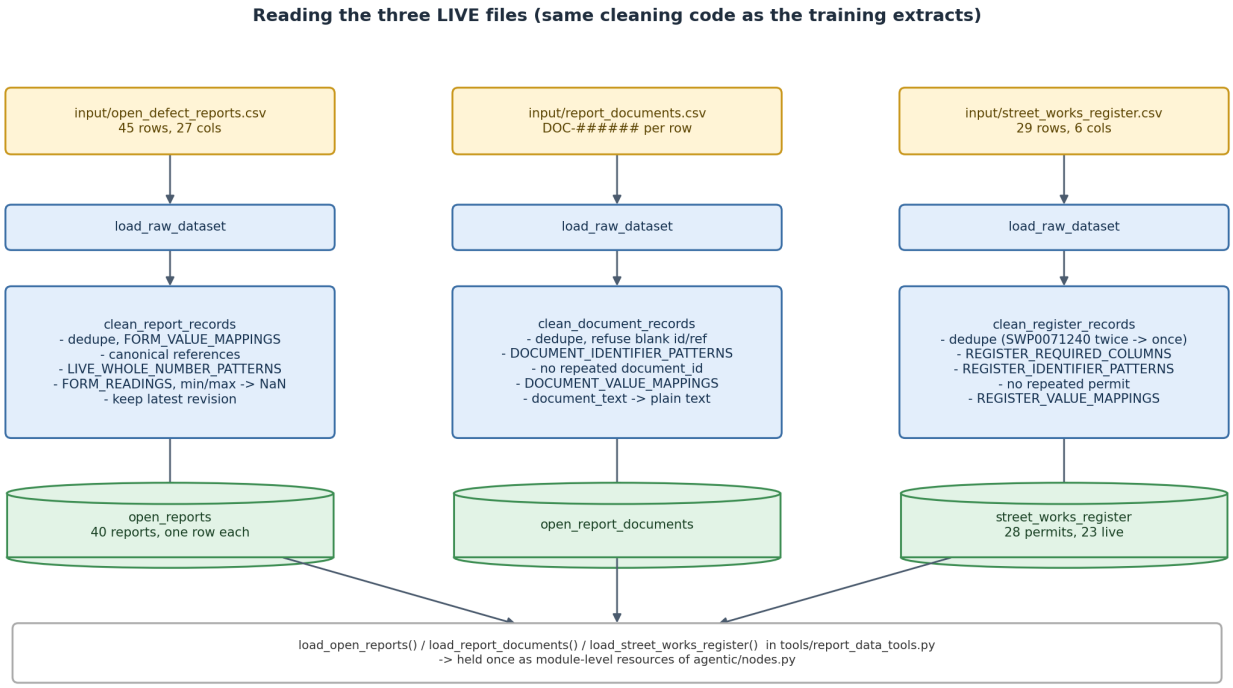


Figure 18 - The live queue, documents and register are cleaned by the same code as the training extracts.

Where the data comes from: data/input/open_defect_reports.csv (45 rows -> 40 open reports at their latest revision), data/input/report_documents.csv, data/input/street_works_register.csv (29 rows -> 28 permits, 23 live; the first two rows are the same permit written twice).

6.2 The report queue order

rank_report_queue puts reports whose defect type **or** linked type is QUEUE_FIRST_DEFECT_TYPE first, then every other report; each group in report-reference order so the oldest is reviewed first. The queue is **never** ordered by a model's figure. A report's status is REPORT_STATUS_AWAITING_REVIEW until a review is stored, REVIEW_STATUS_AWAITING from then, and then the status REVIEW_STATUS_BY_DECISION gives its decision.

What comes out / where it goes: A running FastAPI app on API_HOST:8081 (interactive docs at /docs) holding all resources in memory; database/checkpoint.db ready for paused runs.

7. Reviewing one report - everything the code establishes (no LLM)

Before any agent is called, `gather_report_context(report_reference)` establishes every fact about one report. The same function runs at start-up for the queue and detail pages and again inside the workflow's first node, so what a page shows before review is exactly what the agents are given.

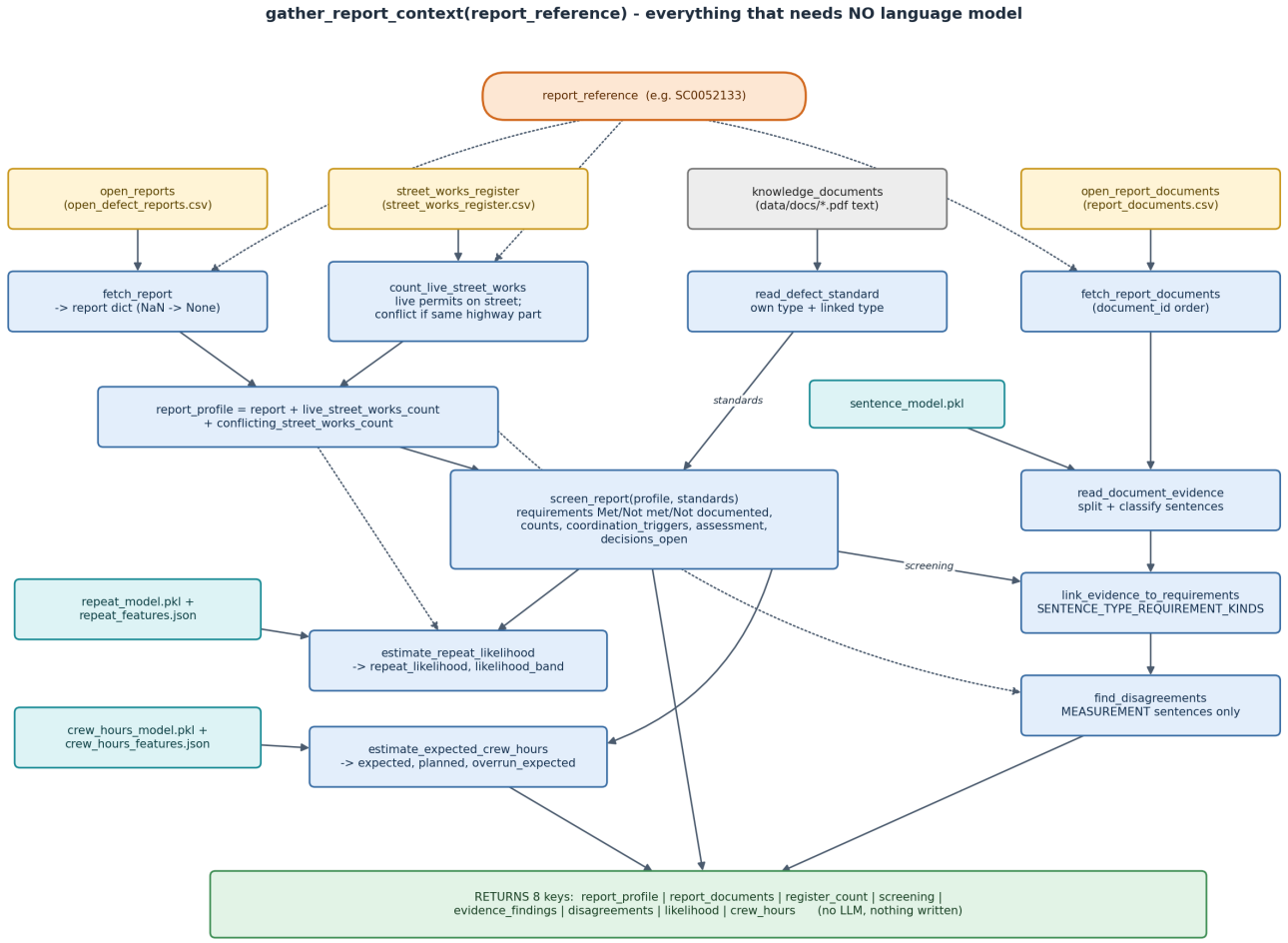


Figure 19 - gather_report_context: eight results, each traced to its source file or model.

Result key	Produced by	Data comes from
report_profile	fetch_report + the two counts	open_reports (open_defect_reports.csv) + register counts
report_documents	fetch_report_documents (document_id order)	open_report_documents (report_documents.csv)
register_count	count_live_street_works	street_works_register.csv: live permits on the street; conflicting = same highway part (DEFECT_TYPE_HIGHWAY_PARTS) as own or linked type
screening	screen_report	report_profile + standards read by read_defect_standard from knowledge_documents (PDFs)
evidence_findings	read_document_evidence + link_evidence_to_requirements	report_documents + sentence_model.pkl + screening
disagreements	find_disagreements	MEASUREMENT findings vs report_profile readings
likelihood	estimate_repeat_likelihood	repeat_model.pkl + repeat_features.json + profile + screening counts/flags
crew_hours	estimate_expected_crew_hours	crew_hours_model.pkl + crew_hours_features.json + profile + screening flags

7.1 Screening each requirement

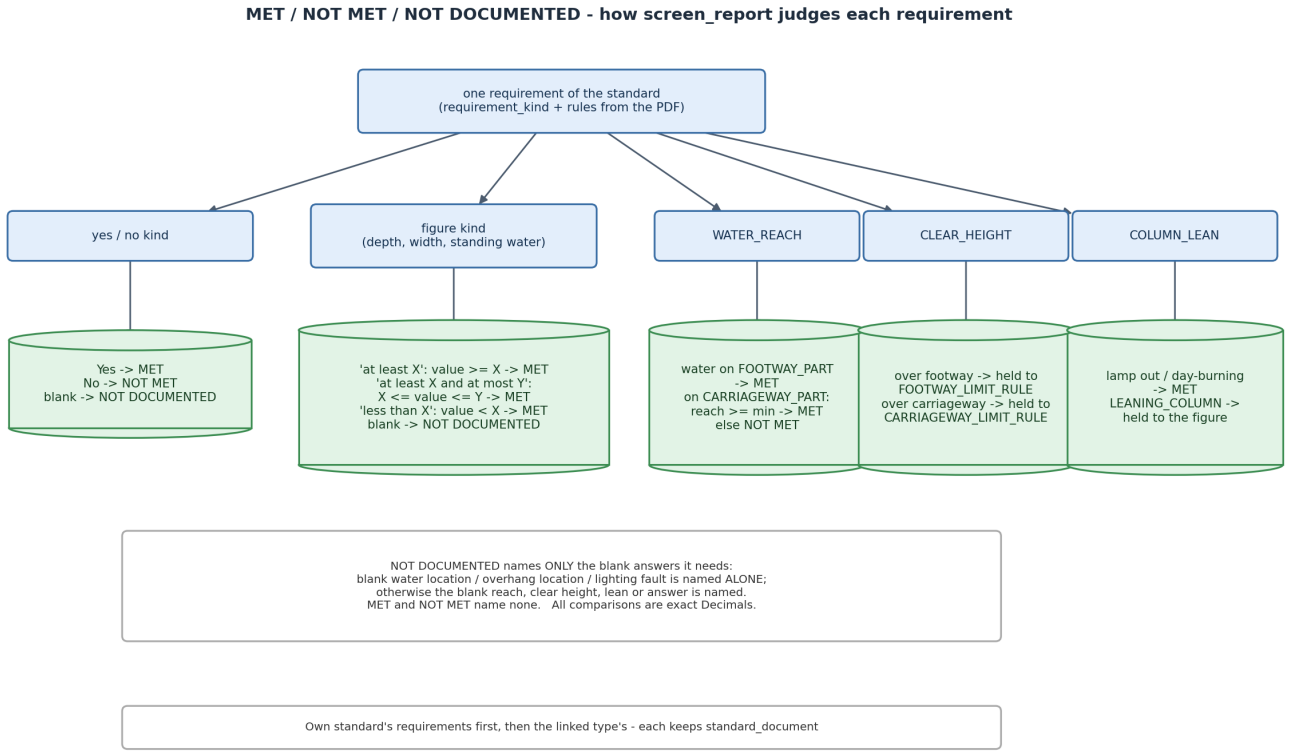


Figure 20 - How screen_report gives each requirement Met, Not met or Not documented.

Each requirement is judged only on the report's answers named by REQUIREMENT_ANSWER_COLUMNS, against the figures of the standard it came from, using exact Decimals (a reach of 1.4 is exactly 1.4). 'At least' is met at its figure; 'at least and at most' at either figure; 'less than' is not met at its figure. A report with a linked type is screened under its own standard first, then the linked type's; each requirement keeps `standard_document`, the PDF it came from.

7.2 Street works, coordination triggers and the assessment

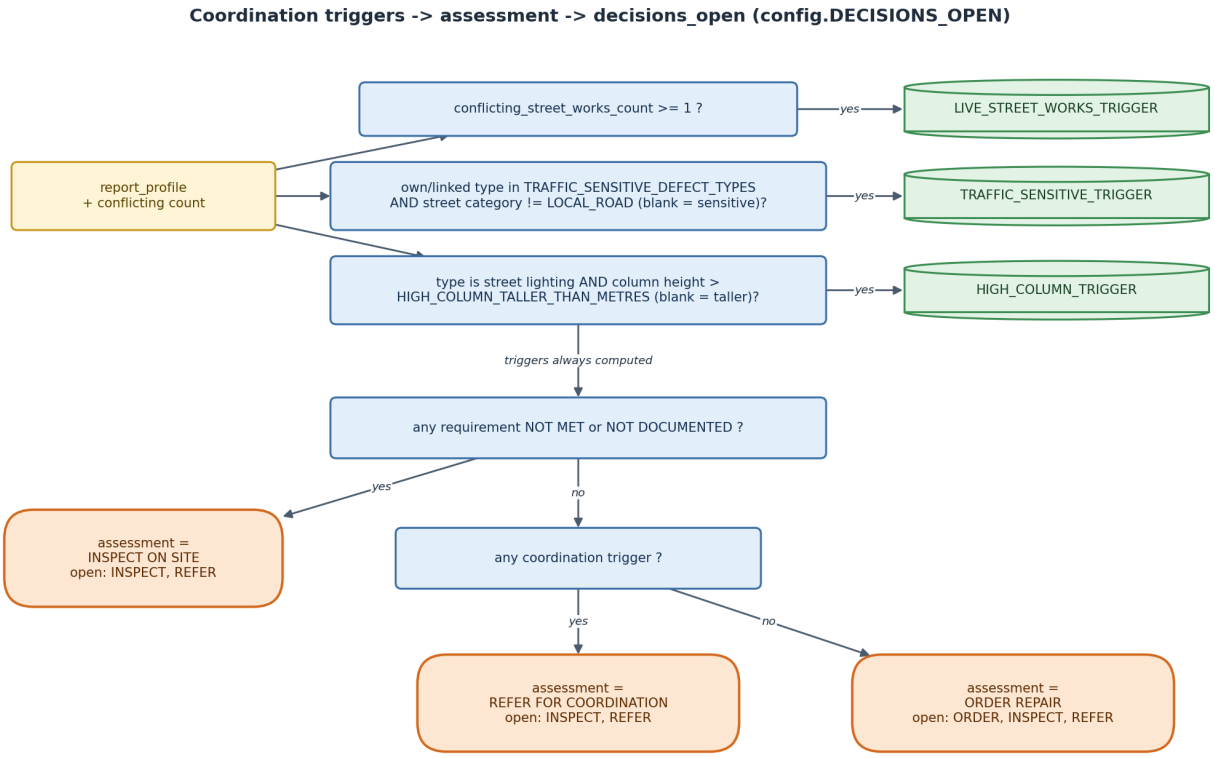


Figure 21 - Coordination triggers are always found; the assessment then follows a fixed priority.

A blank answer never lets a report slip past coordination: a blank street category counts as traffic sensitive and a blank column height counts as taller than HIGH_COLUMN_TALLER_THAN_METRES. Requirements come first: any Not met / Not documented makes the assessment INSPECT ON SITE even if a trigger also applies.

Decision	What it means	Open when
ORDER REPAIR	Repair goes ahead from the report; a works order then a resident update are written	Only when the assessment is ORDER REPAIR
INSPECT ON SITE	An inspector visits and measures first; an inspection request is written	Always (a report is never closed from the desk)
REFER FOR COORDINATION	Goes to the network coordination team with its review and a coordination brief	Always

7.3 Worked example (Report Rules, section 13)

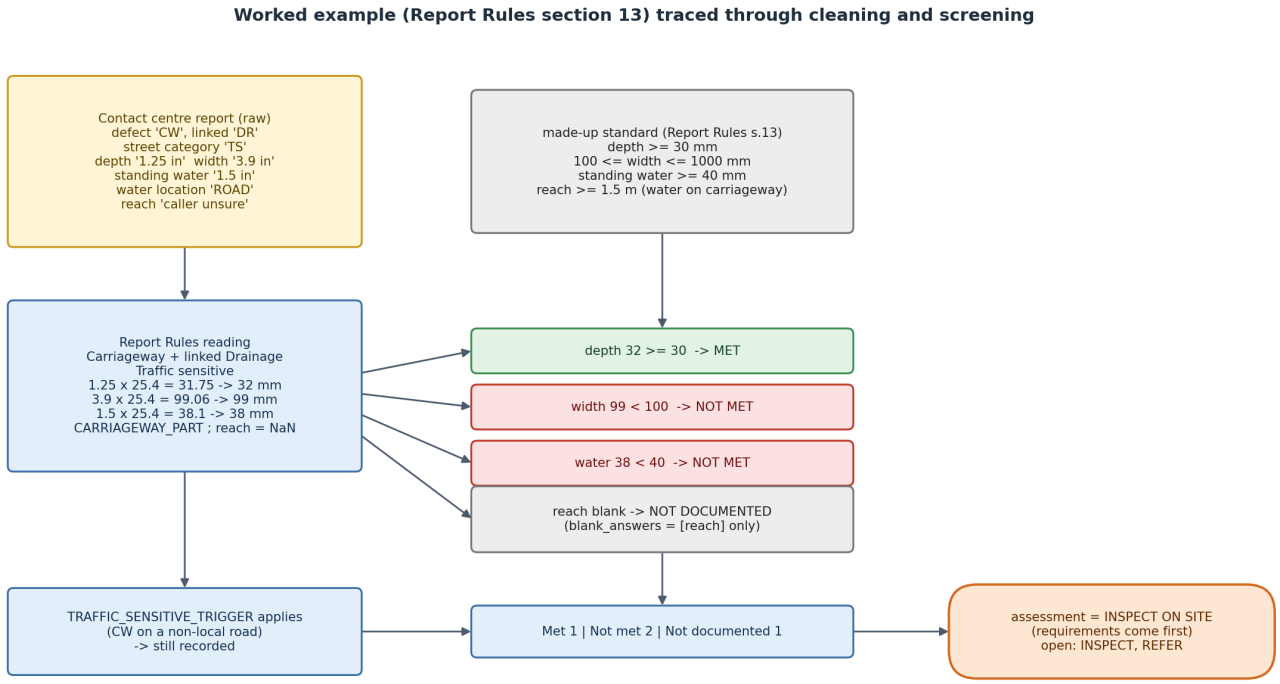


Figure 22 - The specification's worked example traced through cleaning and screening.

A contact-centre report written CW linked DR on a street written TS gives depth 1.25 in (31.75 -> 32 mm: Met), width 3.9 in (99.06 -> 99 mm: Not met), standing water 1.5 in (38.1 -> 38 mm: Not met), water on the ROAD and reach 'caller unsure' (Not documented, asking for the reach alone). Met 1, Not met 2, Not documented 1 -> **INSPECT ON SITE**. The traffic-sensitive trigger also applies, but the requirements come first. (The standard in this example is made-up and belongs to no real defect type.)

7.4 Reading the attached documents

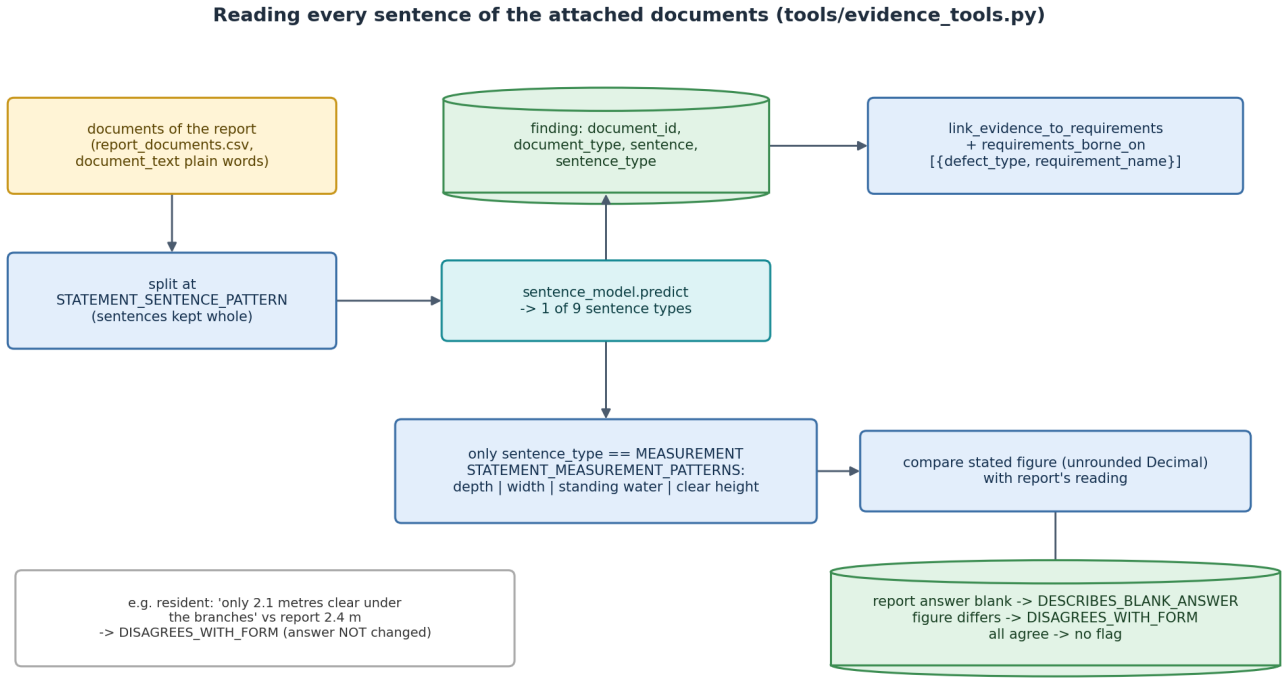


Figure 23 - Every sentence is split, typed by the NLP model, linked to requirements, and measurements are compared.

Each document is split at STATEMENT_SENTENCE_PATTERN, keeping sentences whole. The sentence model gives each one of nine types; SENTENCE_TYPE_REQUIREMENT_KINDS says which requirement kinds each type bears on (hazard, incident, traffic and use, previous reports and background bear on none). Only MEASUREMENT sentences are searched for a depth, width, standing water or clear height. If the report left that answer blank the sentence is flagged DESCRIBES_BLANK_ANSWER; if the figure differs, DISAGREES_WITH_FORM. A flag **never changes** an answer - it is left to the Highways Inspector.

7.5 Turning model output into figures

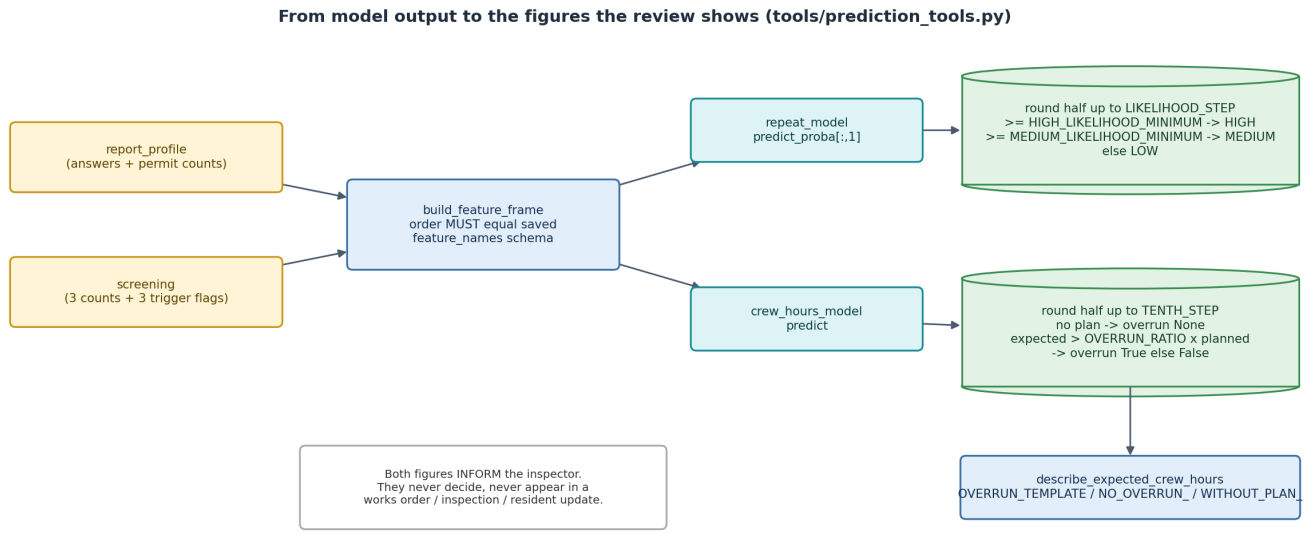


Figure 24 - Repeat likelihood band and expected crew hours / overrun.

The inputs are rebuilt with build_feature_frame and must equal the saved schema in order, otherwise the call is refused. The likelihood is the probability rounded half up to LIKELIHOOD_STEP and banded HIGH / MEDIUM / LOW.

Expected crew hours are rounded to TENTH_STEP; an overrun is expected when they exceed OVERRUN_RATIO x planned hours, and is None when nothing was planned. Neither figure decides anything, and neither ever appears in a works order, inspection request or resident update.

8. The multi-agent LangGraph workflow

A review run is a LangGraph state machine of ten nodes. Five agents review the report before the gate; up to two more write what the decision calls for. Each agent is exactly **one** Gemini call that interprets facts the tools already established. A run makes LLM_CALLS_BEFORE_DECISION calls before the gate and the total LLM_CALLS_BY_DECISION gives for its decision.

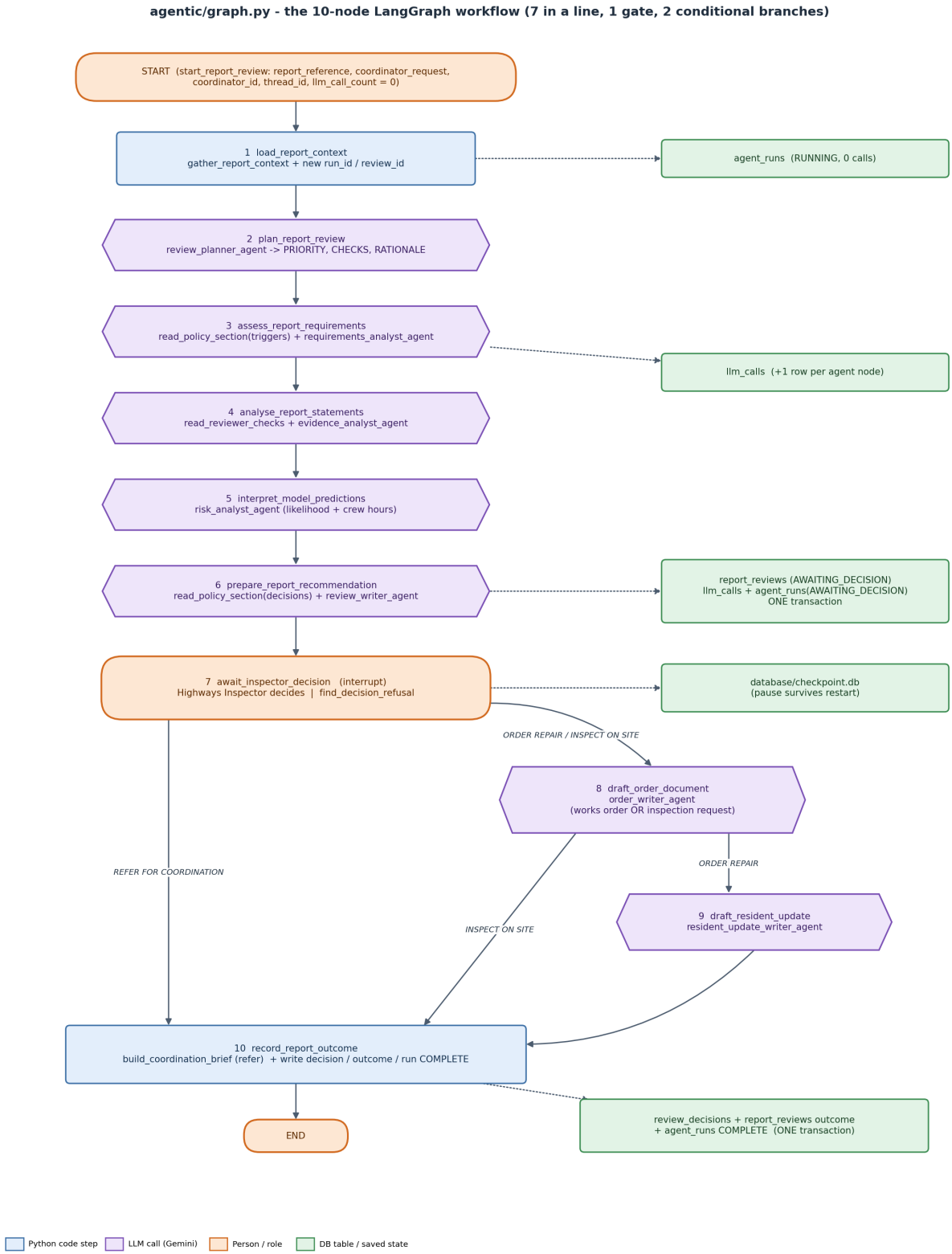


Figure 25 - The ten nodes, the human gate, the two conditional branches and what each node writes.

8.1 What each node reads, does and writes

#	Node	Reads (state keys / sources)	Does	Writes to state	Writes to DB
1	load_report_context	report_reference, coordinator_request, coordinator_id, thread_id	gather_report_context; makes RUN_ID_PREFIX + hex and REVIEW_ID_PREFIX + hex	run_id, review_id + 8 context keys	agent_runs (RUNNING, 0 calls)
2	plan_report_review	report_profile, report_documents, screening, register_count, coordinator_request	plan_review (planner agent)	review_plan, llm_call_count +1	llm_calls
3	assess_report_requirements	screening, profile, register_count, review_plan + Highways Policy triggers section	analyse_requirements	requirements_analysis, +1	llm_calls
4	analyse_report_statements	evidence_findings, disagreements, profile, screening, review_plan + statement guides' reviewer checks	review_evidence	evidence_review, +1	llm_calls
5	interpret_model_predictions	likelihood, crew_hours, profile, screening, review_plan	analyse_predictions	prediction_analysis, +1	llm_calls
6	prepare_report_recommendation	all context + four replies + Highways Policy decisions section	write_review_recommendation	llm_call_count +1	ONE txn: llm_calls + report_reviews (AWAITING) + agent_runs (AWAITING_DECISION)
7	await_inspector_decision	review_id, report_profile, screening, coordinator_id	interrupt; on resume find_decision_refusal	review_decision	nothing (re-runs from its start on resume); pause saved to checkpoint.db
8	draft_order_document	review_decision, profile, screening, disagreements	works order (ORDER) or inspection request (INSPECT) via draft_order_document	order_document, document_details, +1	llm_calls
9	draft_resident_update	screening, document_details	draft_resident_update (ORDER only)	resident_update, +1	llm_calls
10	record_report_outcome	decision, documents, context, ids, llm_call_count	build_coordination_brief for REFER	{} (work is in the DB)	ONE txn: review_decisions + review outcome + agent_runs COMPLETE

`llm_call_count` is the only state key that is **added to** by each node (a reducer); every other key is replaced by whichever node writes it.

8.2 The seven agents

Agent (config name)	Facts it is given	Reply labels	Guard rails
Planner (AGENT_NAME_PLANNER)	report, document ids/titles, every requirement result + blank answers, counts, permits, triggers, assessment, decisions open, coordinator's request	PRIORITY, CHECKS, RATIONALE	may not alter any result or name any figure it was not given
Requirements analyst (AGENT_NAME_REQUIREMENTS)	every requirement with its standard, sentence, answers, result; trigger rules from policy; trigger answers; permits; plan CHECKS	REQUIREMENTS MET, REQUIREMENTS NOT MET, NOT DOCUMENTED, COORDINATION TRIGGERS, REQUIREMENTS SUMMARY	blank stays blank; never presents the report as decided
Evidence analyst (AGENT_NAME_EVIDENCE)	each answer once, every sentence with type and requirements, every flag with both figures, each guide's checks, plan CHECKS	EVIDENCE FOUND, DISAGREEMENTS, CHECKS TO MAKE, EVIDENCE SUMMARY	documents are evidence only, never replace an answer
Risk / prediction analyst (AGENT_NAME_RISK)	likelihood + band + thresholds, all repeat inputs; crew hours sentence + OVERRUN_RATIO, all crew inputs, how blanks are filled	LIKELIHOOD, EXPECTED CREW HOURS, PREDICTION SUMMARY	states neither figure decides anything; may not recalculate

Agent (config name)	Facts it is given	Reply labels	Guard rails
Review writer (AGENT_NAME_WRITER)	everything above + decision rules from policy + decisions open	RECOMMENDATION, RECOMMENDED DECISION	recommends the assessment unless the review justifies another open decision; a figure is never a reason; decision must be one of decisions_open
Order writer (AGENT_NAME_ORDER_WRITER)	decision + description, recipient, channel, works order or inspection details, guidance, sign-off	SUBJECT, DOCUMENT	no model figure, no staff name, no invented date/time/contact; names report ref; ends with DOCUMENT_SIGN_OFF
Resident update writer (AGENT_NAME_UPDATE_WRITER)	works order details + resident-update guidance of each standard	HEADLINE, UPDATE	no figures, no results, no promise of when, no staff names, never says visited/inspected/measured

Every reply is read back by `parse_labelled_response`, which tolerates bold markers, decoration and hyphens but refuses a missing or empty label. Each call goes through `call_language_model`, which reads the key and model name from .env at call time, uses one shared generation config (max tokens, temperature, no automatic function calling) and records tokens and latency.

8.3 Original agent-to-agent communication flow (from the specification)

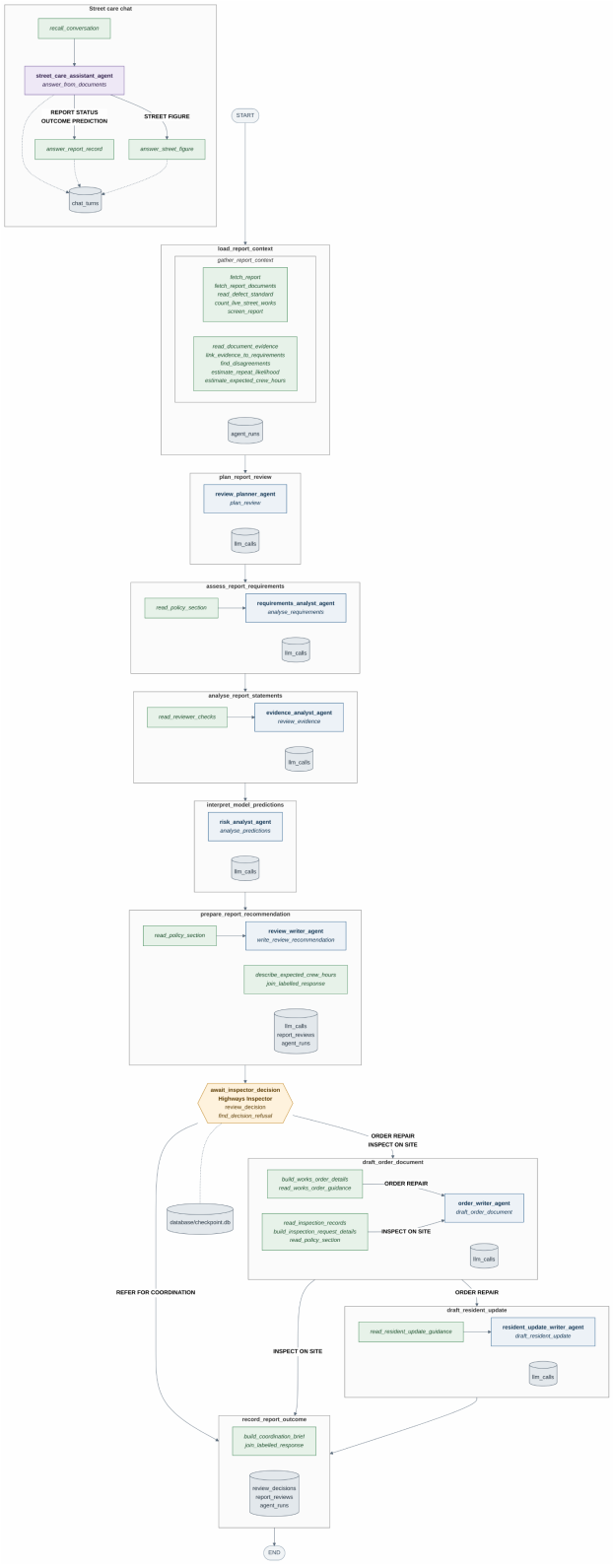


Figure 26 - Agent-to-agent communication flow, as drawn in the project specification.

8.4 Opening a review - sequence

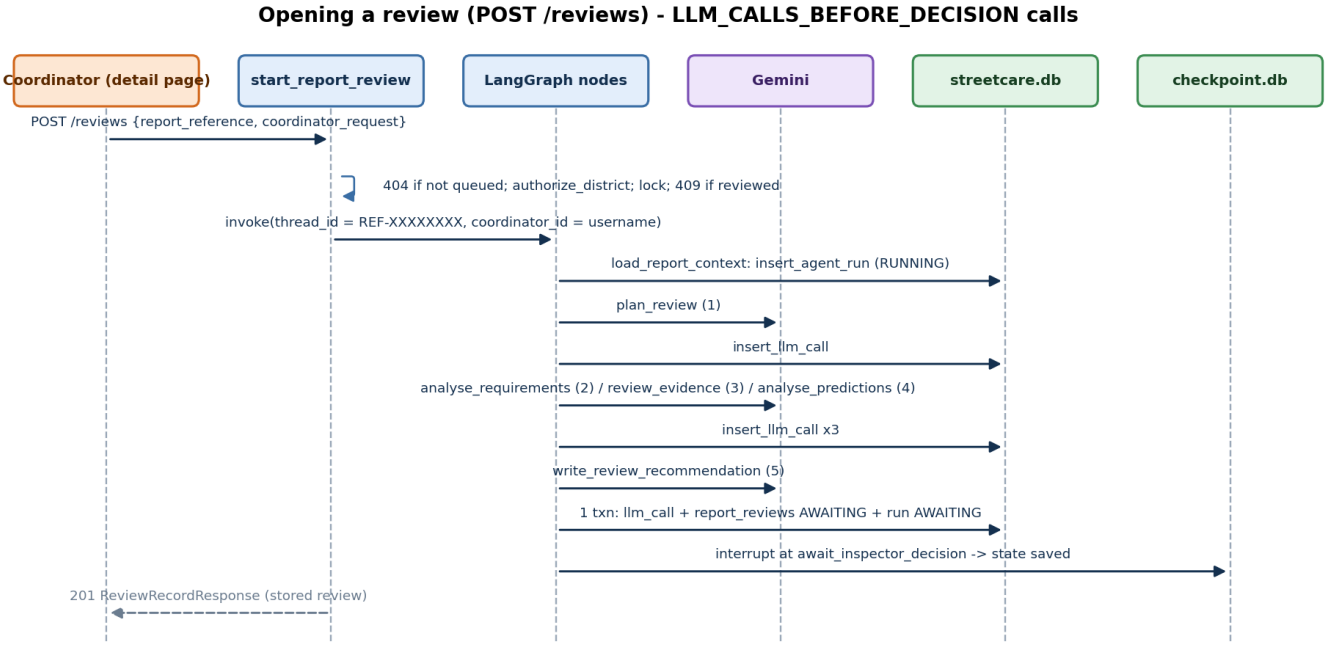


Figure 27 - POST /reviews: from the coordinator's click to a review waiting at the gate.

8.5 The human gate - every check before a decision

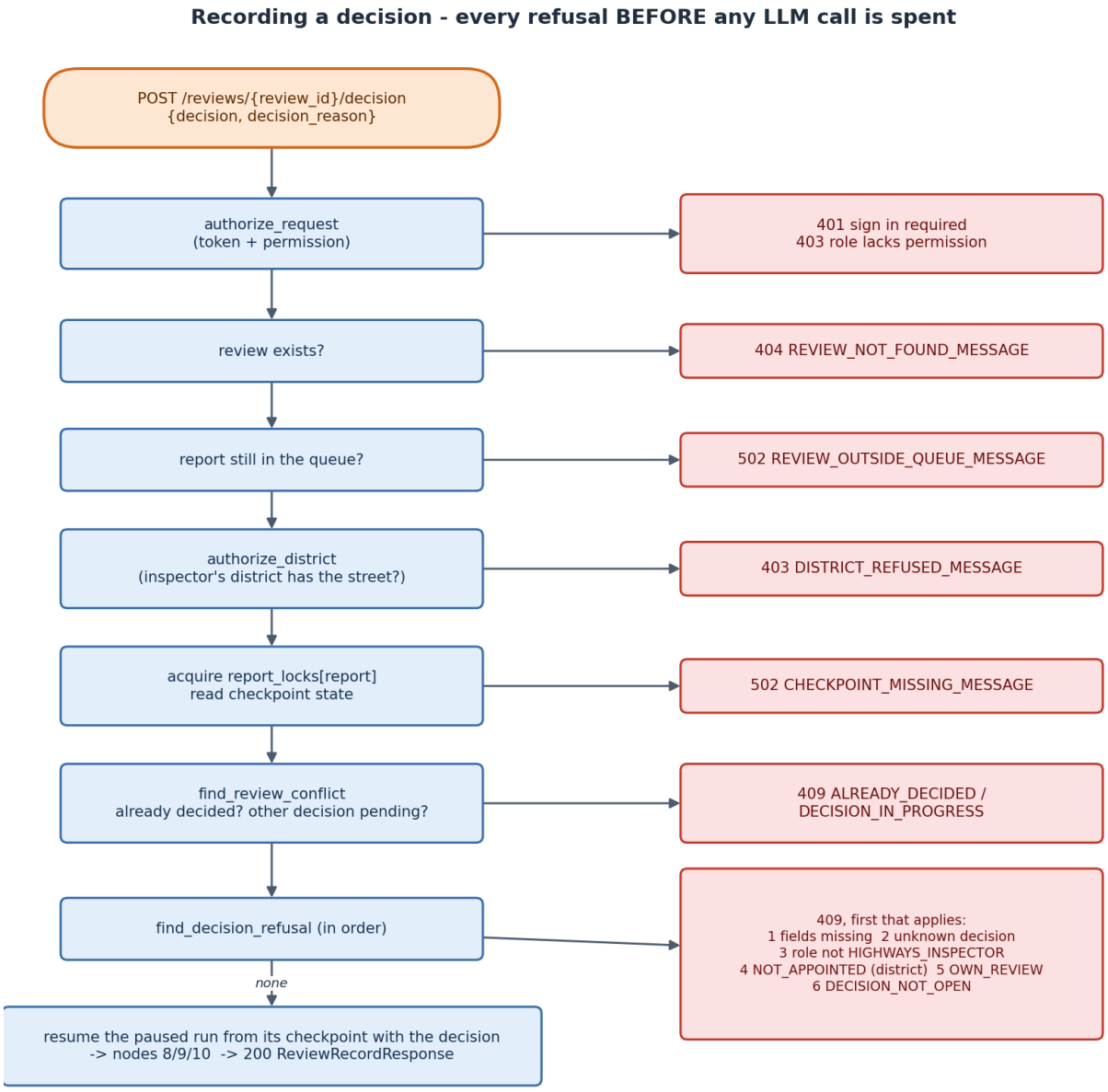


Figure 28 - All refusals are given before the run resumes and before any LLM call is spent.

The district rule means a Highways Inspector can only act on streets of their own district (INSPECTION_DISTRICT_STREETS). The own-review rule means the person who opened the review (the run's coordinator_id) can never decide it - which is why two inspectors are appointed per district. If the same decision is sent again after a failure it finishes the run; a different decision while one is pending gets DECISION_IN_PROGRESS.

8.6 Deciding a review - sequence

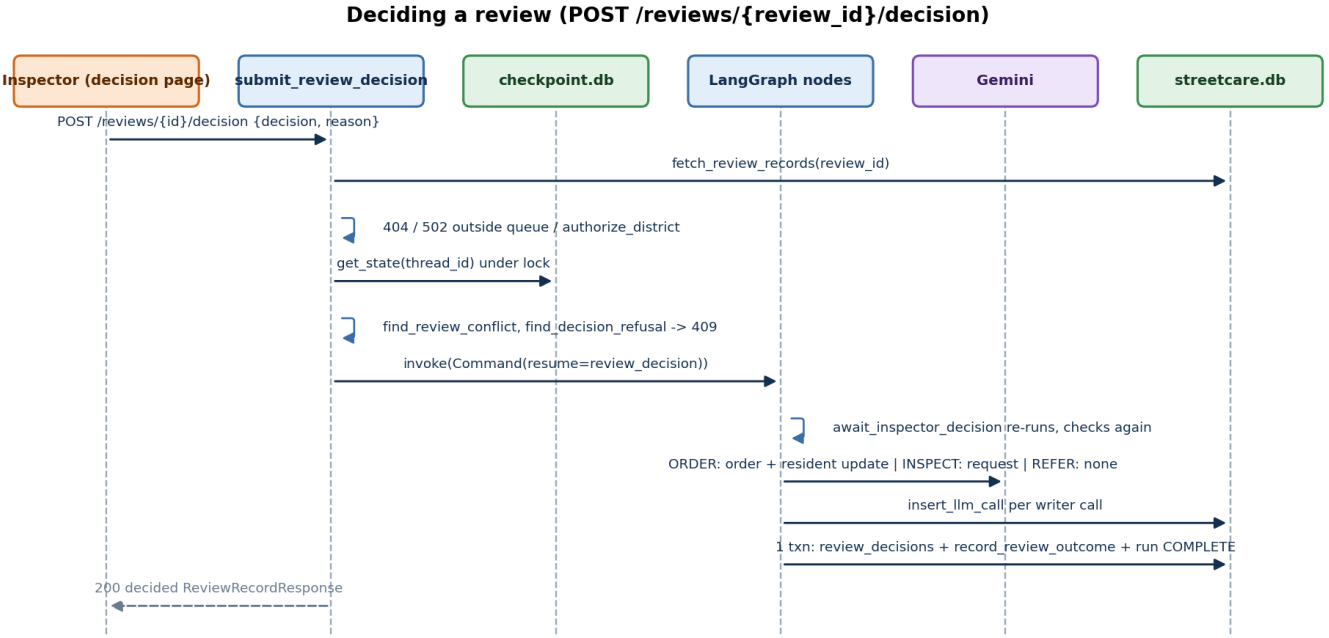


Figure 29 - POST /reviews/{review_id}/decision: resuming the paused run from its checkpoint.

8.7 What each decision writes

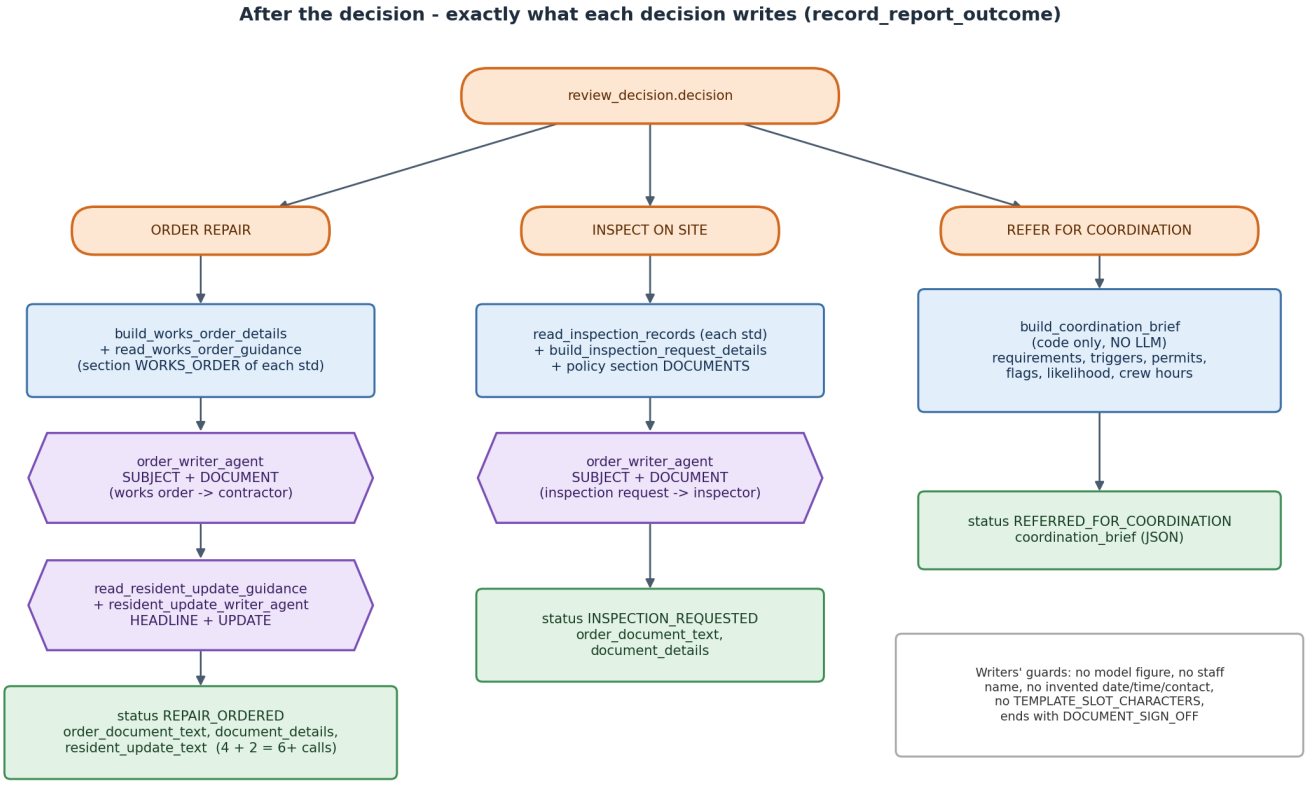


Figure 30 - ORDER REPAIR, INSPECT ON SITE and REFER FOR COORDINATION each lead to different documents.

Decision	Facts built by code	Written by LLM	Stored on report_reviews
ORDER REPAIR	build_works_order_details: reference, WORKS_ORDER_DETAIL_COLUMNS, each type's WORKS_ORDER_MEASUREMENT_COLUMNS; guidance = section WORKS_ORDER of each standard	works order (to the repair contractor), then resident update	status REPAIR_ORDERED, order_document_text, document_details, resident_update_text

Decision	Facts built by code	Written by LLM	Stored on report_reviews
INSPECT ON SITE	build_inspection_request_details: every requirement not Met with answers and blank answers, what each standard says an inspection records, every flagged statement; guidance = policy documents section	inspection request (to the inspector)	status INSPECTION_REQUESTED, order_document_text, document_details
REFER FOR COORDINATION	build_coordination_brief: details, requirements, triggers, live + conflicting permits, flags, likelihood, crew hours	nothing	status REFERRED_FOR_COORDINATION, coordination_brief (JSON)

9. The REST service, access control and the dashboard

9.1 Life of every request

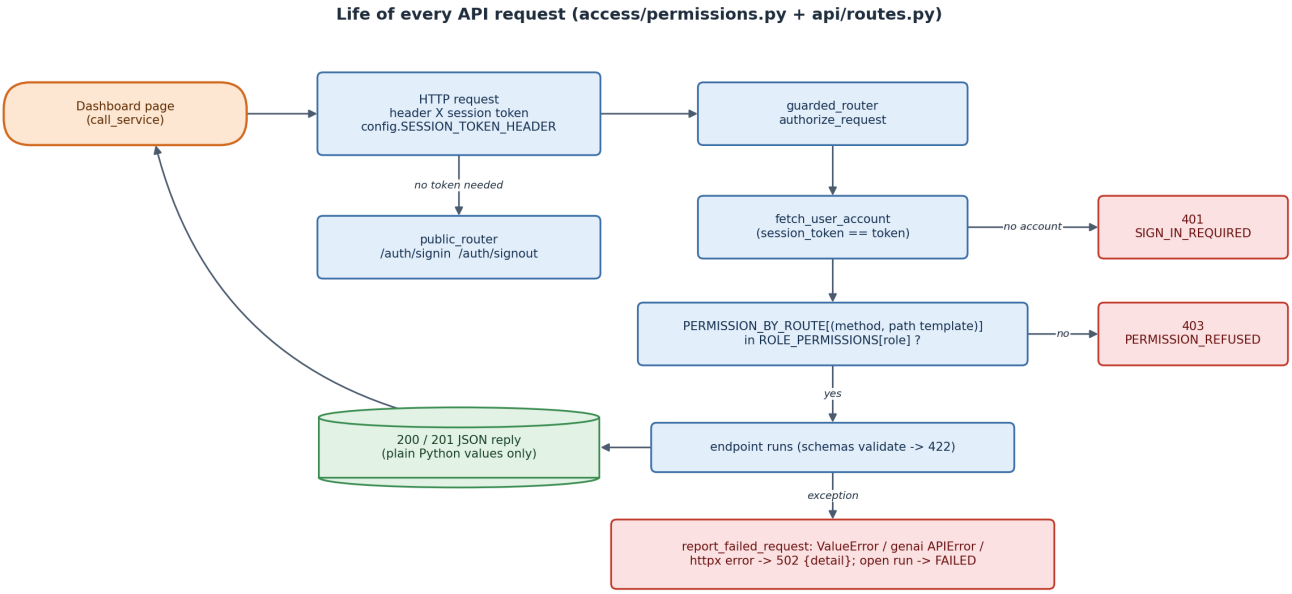


Figure 31 - Token check, permission check, endpoint, and one failure handler.

Signing in and out sit on `public_router`; every other endpoint is on `guarded_router`, included behind `authorize_request` so the guard cannot be forgotten. The permission needed is looked up from PERMISSION_BY_ROUTE by method and path template (e.g. /reviews/{review_id}/decision) and checked against ROLE_PERMISSIONS - refused with 403 before the endpoint runs, whether or not the thing asked for exists. Any ValueError, Gemini APIError or httpx error becomes a 502 with the failure's own message, and a run the failing request opened is marked FAILED with its call count.

9.2 Signing in

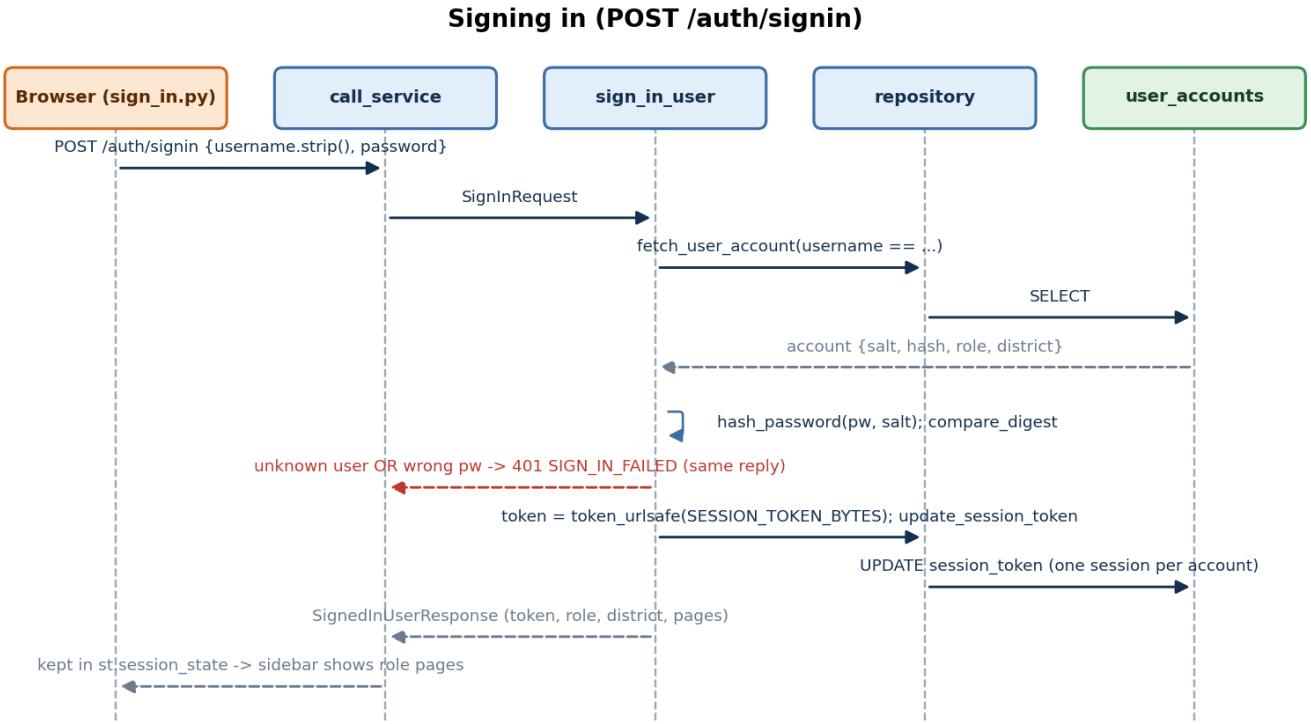


Figure 32 - POST /auth/signin: identical 401 for unknown user or wrong password; a fresh token replaces any old one.

9.3 Endpoints

Endpoint	Function	Data comes from	Returns / writes
POST /auth/signin	sign_in_user	user_accounts	SignedInUserResponse; stores new session token
POST /auth/signout	sign_out_user	token header	clears token
GET /queue?defect_type=	list_report_queue	report_queue (built at start-up) + stored review statuses	QueueRow list in queue order (filter by own or linked type)
GET /queue/{report_reference}	get_report_detail	report_details + review status	ReportDetailResponse (404 if not queued)
POST /reviews	start_report_review	ReviewRequest + workflow	201 ReviewRecordResponse; 5 LLM calls; 409 if latest revision already reviewed
GET /reviews?status_value=	list_report_reviews	report_reviews	reviews of the asked statuses
POST /reviews/{id}/decision	submit_review_decision	DecisionRequest + checkpoint	decided review; writer calls
GET /reviews/{id}/calls	get_review_calls	llm_calls + checkpoint	calls vs expected_call_count, total tokens, total latency
POST /chat/ask	ask_street_care_assistant	chat_turns, chroma store, PDFs, review records, report history	ChatResponse; inserts chat_turns
GET /chat/turns?turn_limit=	list_own_chat_turns	chat_turns (own)	newest first
GET /audit/chat-turns?turn_limit=	list_recent_chat_turns	chat_turns (all)	newest first

Only POST /reviews, POST /reviews/{review_id}/decision and POST /chat/ask may cost language model calls. Every reply carries only plain Python values; a malformed request is refused with 422 before any work is done.

9.4 Roles and pages

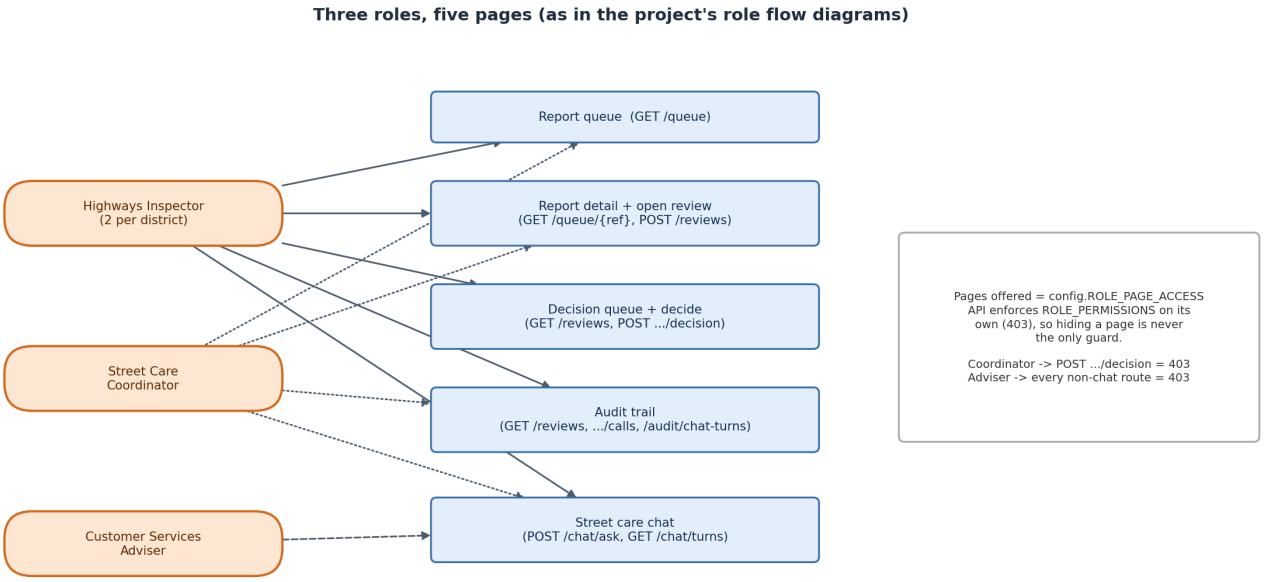


Figure 33 - Which role reaches which page.

Original role flow diagrams (from the specification)

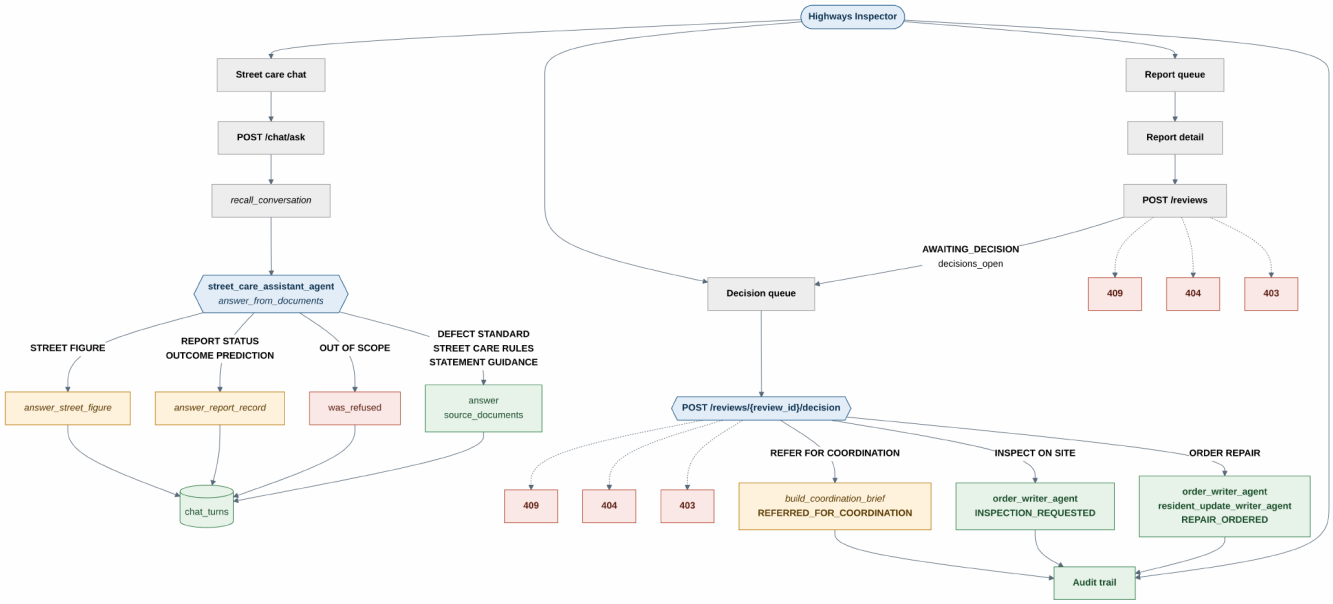


Figure 34 - Role flow - Highways Inspector.

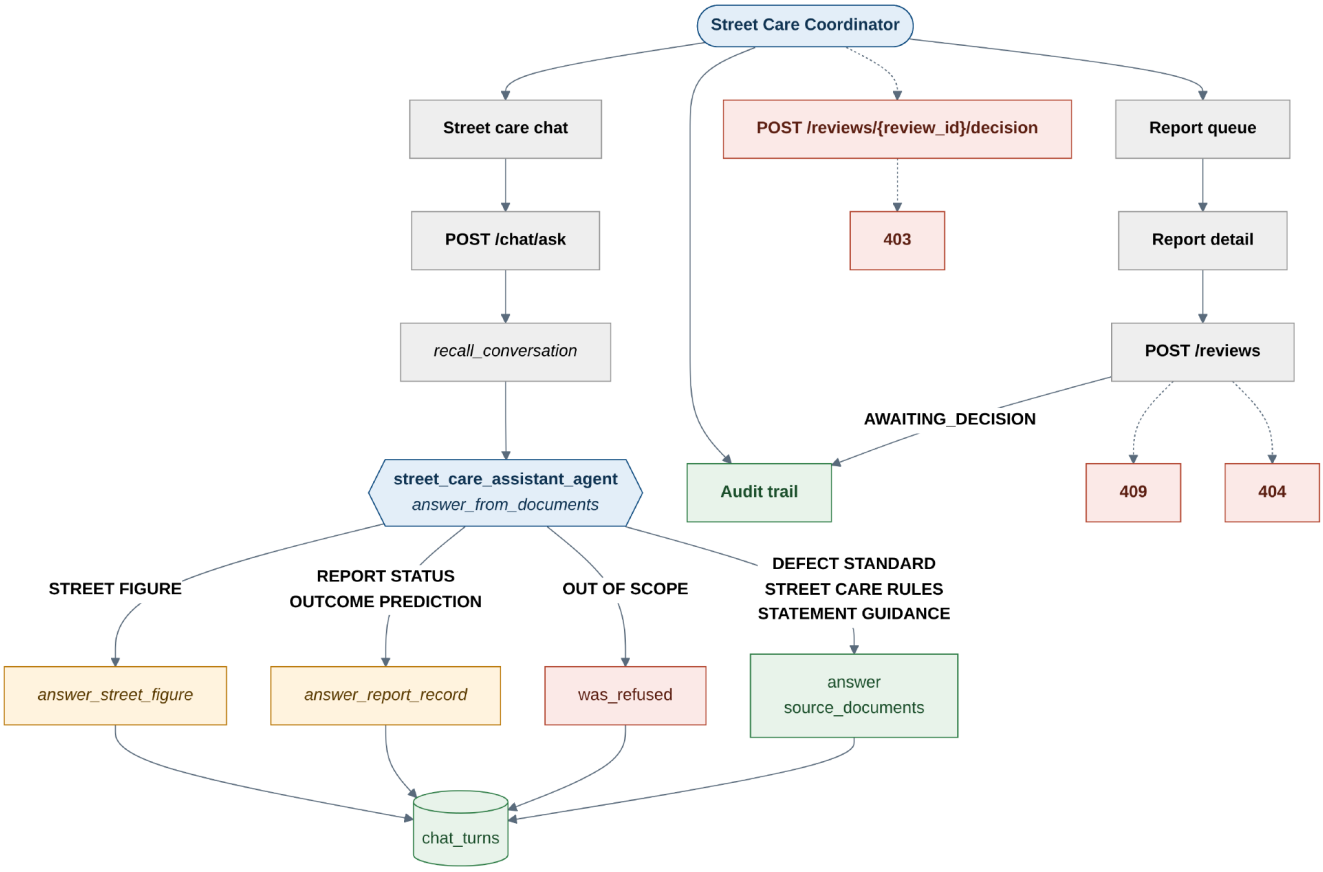


Figure 35 - Role flow - Street Care Coordinator (decision endpoint refused with 403).

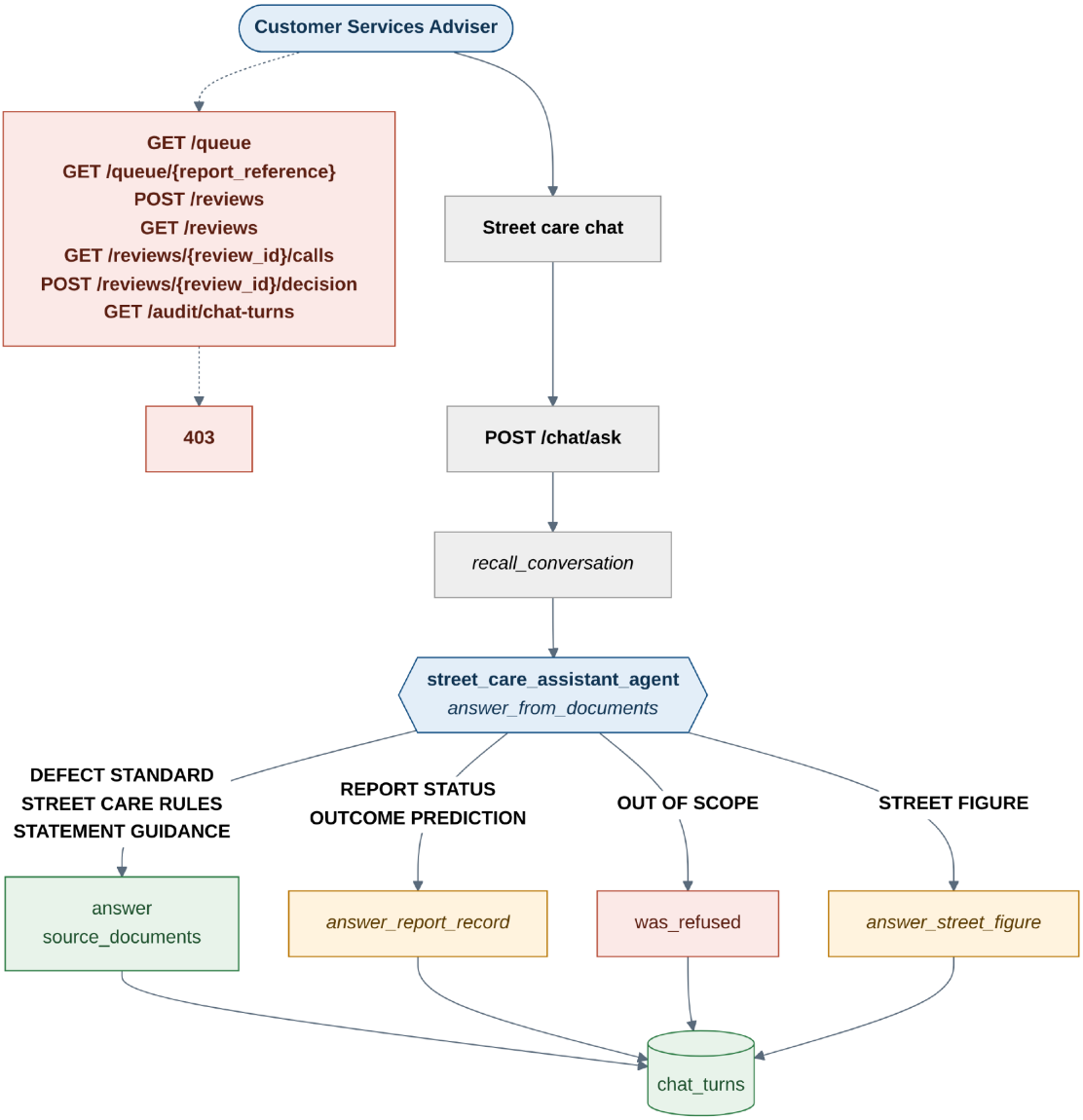


Figure 36 - Role flow - Customer Services Adviser (only the chat; every other route 403).

9.5 The dashboard pages

The dashboard (Streamlit, port 8080) holds **no business rule**: every figure it shows comes from the service through `dashboard/api_client.call_service`, which adds the session token, shows a spinner only for calls that cost LLM calls, returns to sign-in on a 401, and shows the service's own detail on any error.

Page	Shows	Data from
Sign in	title, subtitle, username/password form, 'session ended' notice	POST /auth/signin
Report queue	top counts (reports, awaiting review / decision, queue-first type, each assessment); filter; table with position, reference, revision, types, street, district, channel, status, assessment, 3 counts, permits, triggers, flags, documents, likelihood + band, expected crew hours, overrun	GET /queue
Report detail	headline + counts; tabs: screening (requirements, decisions open, triggers, permits), statement evidence (every sentence, flags), report (answers labelled by FORM_ANSWER_LABELS), review (recommendation + decision); start-review form	GET /queue/{ref}, GET /reviews, POST /reviews
Decision queue	inspector's district + streets; reviews AWAITING on those streets; recommendation, figures, requirements, triggers, permits, flags, agent write-ups; decision form limited to decisions_open (hidden on own review)	GET /reviews, POST /reviews/{id}/decision

Page	Shows	Data from
Audit trail	reviews by status; full record, documents, coordination brief, call budget (calls vs expected, tokens, latency); latest chat turns from everyone	GET /reviews, GET /reviews/{id}/calls, GET /audit/chat-turns
Street care chat	own conversation (badged by type, with sources); ask box	GET /chat/turns, POST /chat/ask

10. The Street Care chat

One question, one language model call, answered from Wrenford's documents and nothing else. The model classifies the question; anything about a single report or a street is then answered **by code** from the records, within what the asker's role allows.

The Street Care chat - one question, one LLM call, records answered by code

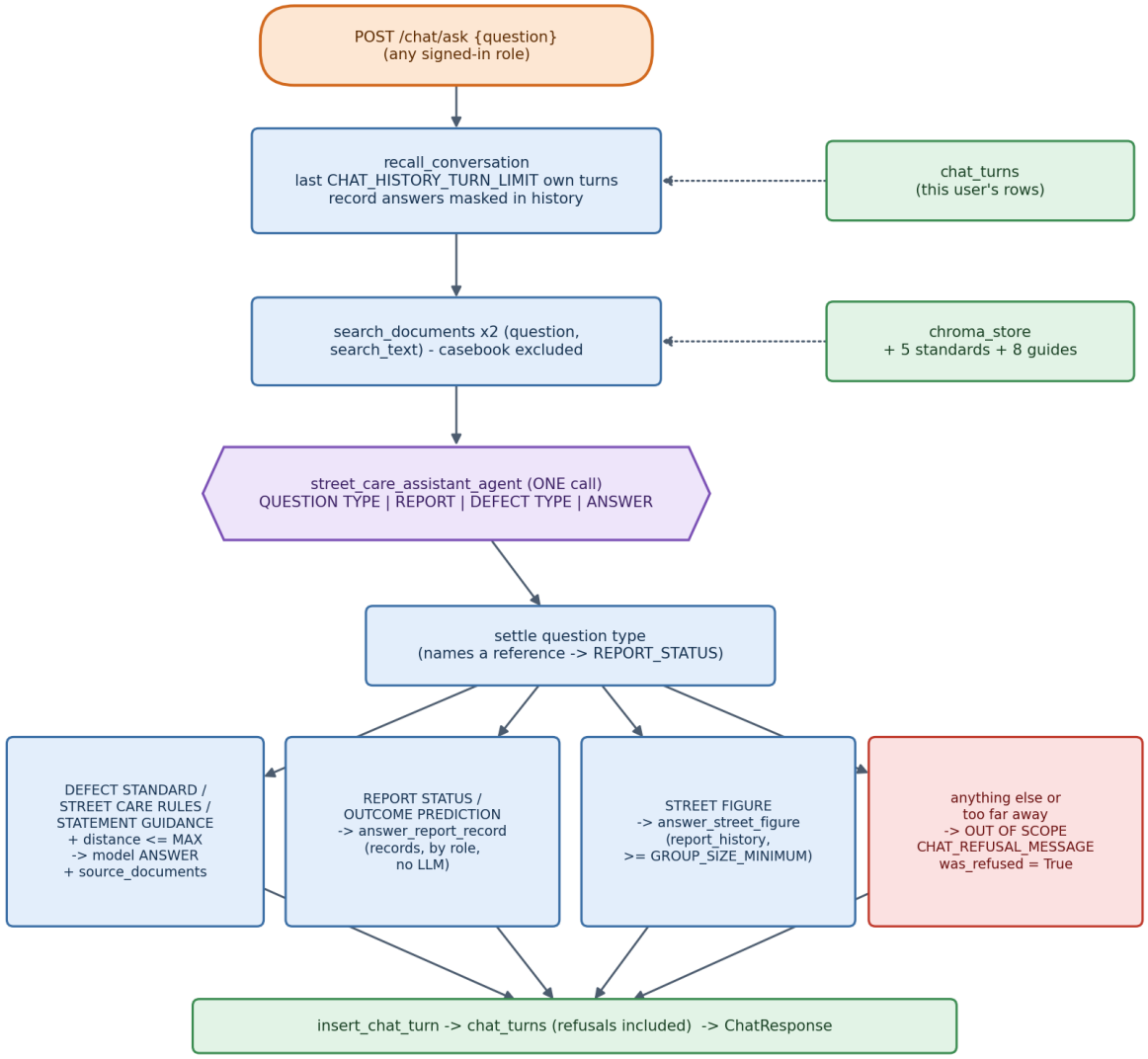


Figure 37 - ask_street_care_assistant: recall, search, one LLM call, routing, recording.

Where the data comes from: chat_turns (the asker's own last CHAT_HISTORY_TURN_LIMIT turns; record answers are masked as CHAT_RECORD_ANSWER_IN_HISTORY), database/chroma_store (two searches: the question and search_text), the five standards' requirements and inspection sections, every statement guide's reviewer checks, report_reviews (for report questions), report_history (for street figures).

Step by step

- **recall_conversation** - history_text (oldest first), search_text (previous question + new one), previous_question, previous_street. Nobody else's turns are ever recalled.
- **answer_from_documents** - two searches together, passages offered once each; one call as AGENT_NAME_STREET_CARE_ASSISTANT replying QUESTION TYPE, REPORT, DEFECT TYPE, ANSWER. A question naming a report reference becomes REPORT_STATUS.

- **Answered** only if the type is one of CHAT_ANSWERED_TYPES and the nearest passage is within CHAT_MAX_RELEVANCE_DISTANCE; source_documents lists the standards / guides / passages used.
- **Report questions** (CHAT_REPORT_TYPES) are replaced by answer_report_record; **street figure** questions by answer_street_figure. Everything else is a refusal (OUT OF SCOPE, CHAT_REFUSAL_MESSAGE).
- **Recorded** - every reply, refusals included, becomes one chat_turns row. A request that fails records nothing.

10.1 What each role may hear about a report

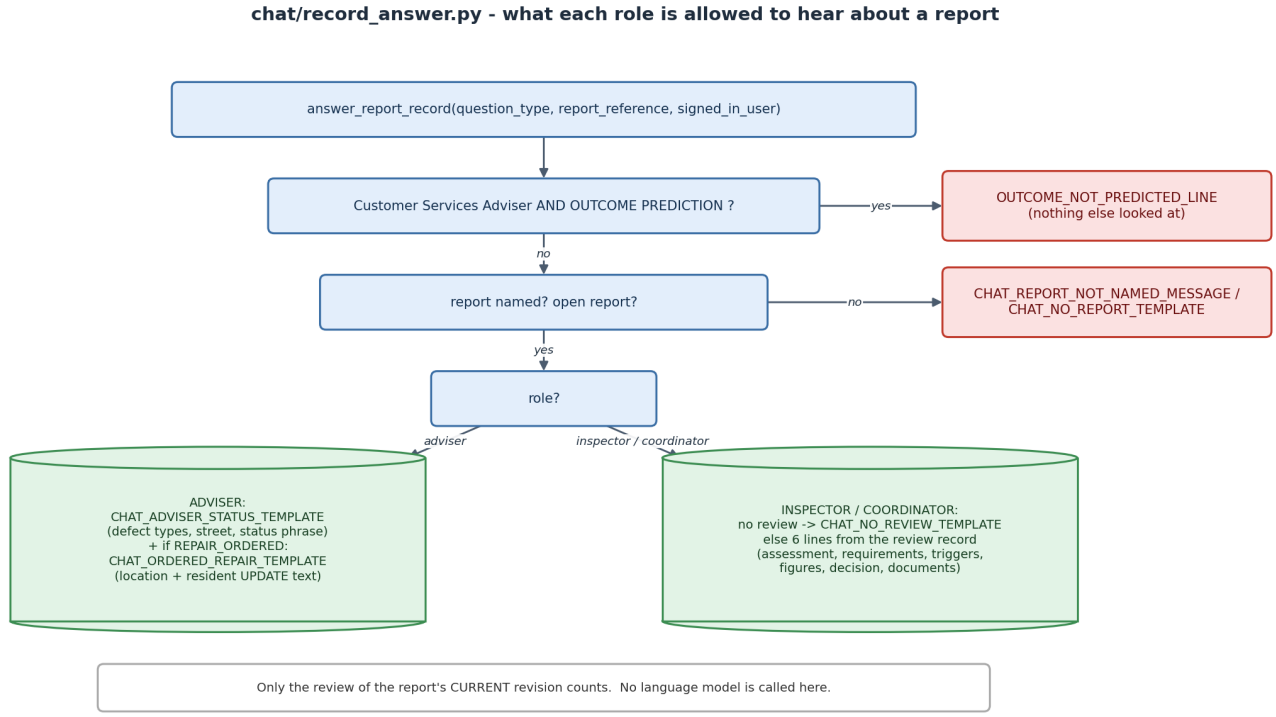


Figure 38 - answer_report_record: advisers never hear a prediction; inspectors and coordinators get the review record.

10.2 The figure line (privacy)

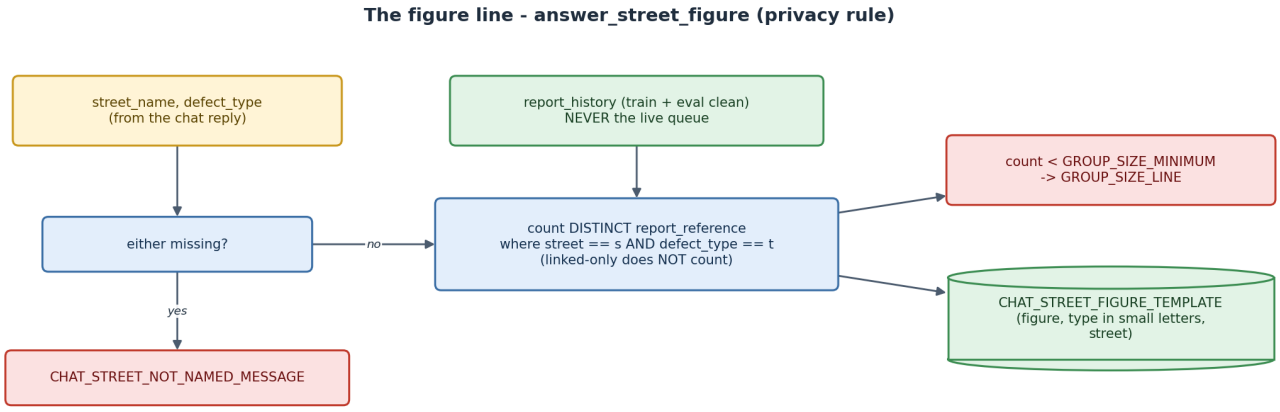


Figure 39 - A street figure is given only when at least GROUP_SIZE_MINIMUM reports make it up.

A figure about a street that fewer than five reports make up would point at the household that made the report, so below GROUP_SIZE_MINIMUM everyone hears GROUP_SIZE_LINE. The count is distinct report references on that street with that defect type (linked-only does not count), from the decided report history - never from the live queue. The answer is the same whoever asks and no language model is involved.

11. End-to-end data lineage - every step in one table

This table follows a single piece of data from the moment it is a raw CSV cell or a PDF sentence to the moment it appears on the dashboard. Use it as the script for explaining the project to the team.

#	Step	Data in (from where)	What happens	Data out (to where)
1	Seed accounts	config.SEED_USER_ACCOUNTS	salt + PBKDF2 hash	user_accounts
2	Read raw extracts	data/*.csv	text only, trim, NULL_TOKENS -> NaN	pandas DataFrame (memory)
3	Clean history	report_history_{train,eval}.csv	dedupe, mappings, references, units, min/max, latest revision, drop no answer	processed/report_history_*_clean.csv
4	Clean repair logs	repair_logs_{train,eval}.csv	dedupe, refuse blank / repeated refs, units, limits, drop no crew hours	processed/repair_logs_*_clean.csv
5	Clean sentences	statement_sentences.csv	ids, mappings, markup -> plain, capitals -> lower, too-short dropped	processed/statement_sentences_clean.csv
6	Quality report	raw + clean frames	profile + detail profile per extract	output/data_quality_report.json
7	Read standards	data/docs/ (5 standards)	sections -> requirements + Decimal figures	standards dict (memory)
8	Features	clean history / logs + standards	requirement counts, trigger flags, feature frame	feature frames (memory)
9	Train + evaluate	feature frames, sentences	LogReg / RandomForest / TF-IDF+LogReg; held-out scores; minimum gate	output/models/*.pkl, *_features.json, model_metrics.json
10	Index documents	data/docs/*.pdf (16)	extract, 136 sections, chunks with overlap, embed	database/chroma_store/
11	Service start	all artefacts + data/input/*.csv	file check; load models, live files, docs, store; build workflow	in-memory resources; checkpoint.db
12	Clean live files	open_defect_reports, report_documents, street_works_register	same cleaning code as training	open_reports, open_report_documents, street_works_register
13	Report context	live tables + standards + 3 models	fetch, count permits, screen, triggers, assessment, evidence, flags, likelihood, crew hours	report_profiles / report_details / queue; later workflow state
14	Sign in	username + password	hash check, fresh token	user_accounts.session_token; browser session
15	View queue / detail	GET /queue, /queue/{ref}	pre-computed rows + stored statuses	dashboard tables
16	Open review	POST /reviews + coordinator request	node 1 (context, run row), nodes 2-6 (5 agents)	agent_runs, llm_calls x5, report_reviews AWAITING
17	Pause	node 7 interrupt	state serialised	database/checkpoint.db
18	Decide	POST /reviews/{id}/decision	all refusals checked; resume from checkpoint	review_decision in state
19	Write documents	decision + profile + screening + PDF sections	works order + resident update / inspection request / coordination brief	llm_calls; review outcome columns
20	Close run	state	one transaction	review_decisions, report_reviews status, agent_runs COMPLETE
21	Audit	GET /reviews, /calls, /audit/chat-turns	read records	audit trail page
22	Chat	question + own history + store + records	one LLM call, code answers records / figures	chat_turns; chat page

12. Running it and what you should see

Entry point	Expected console output (shape)
run_database_setup.py	database: database/streetcare.db; tables: agent_runs, chat_turns, llm_calls, report_reviews, review_decisions, user_accounts; one line per account created; accounts created: 16, already present: 0 (second run: 0 / 16)
run_data_pipeline.py	report_history_train 32738 -> 30569 (2169 removed); report_history_eval 8202 -> 7648 (554); repair_logs_train 26095 -> 25537 (558); repair_logs_eval 6476 -> 6337 (139); statement_sentences 36661 -> 35717 (944); quality report written to output/data_quality_report.json
run_model_training.py	repeat held-out ROC AUC (train, gap); crew hours held-out R2 + MAE (train, gap); sentence held-out macro F1 (train, gap); metrics written to output/model_metrics.json; every model reaches its minimum
run_knowledge_indexing.py	16 documents, 136 sections, NNN passages indexed in database/chroma_store; then each standard's inspection-records section read back under its defect type
run_api_service.py	Starting StreetCare AI API on port 8081. Interactive documentation: http://localhost:8081/docs ... Uvicorn running on http://0.0.0.0:8081
run_dashboard.py	Local URL: http://localhost:8080

Quick talking points for the team

- **Data enters** as 5 messy training CSVs, 3 live CSVs and 16 PDFs - nothing else.
- **Cleaning** makes three channels speak one language (references, units, yes/no, missing) and keeps only the latest revision of each report.
- **Rules** (requirements and their figures) are read from each defect type's own PDF at run time - never typed into code.
- **Three ML models** add context only: repeat likelihood, expected crew hours, and the type of each document sentence.
- **Five agents** explain established facts and recommend; **a Highways Inspector of the right district, who did not open the review, decides.**
- **The pause survives a restart** because LangGraph checkpoints to database/checkpoint.db.
- **Every run, call, review, decision and chat turn** is auditable in database/streetcare.db, with tokens and latency for each call.
- **Privacy:** advisers never hear predictions; street figures need at least five reports.